Topic 2: Synthesis of continuous time control systems. The gain and phase margin. Linear systems and their description in time- and frequency domains. Signal transfer in control systems. Describe the protocol data units (PDU) of the TCP and UDP transport layer protocols and the characteristics and differences of their mechanisms.

1. Synthesis of Continuous-Time Control Systems

Control system synthesis is the process of designing a controller to meet desired performance specifications. The goal is to create a system that is stable, responsive, and accurate.

Key Steps:

- **Modeling**: Create a mathematical model of the plant using differential equations or transfer functions.
- **Specification**: Define requirements (settling time, overshoot, steady-state error).
- **Controller Design**: Choose a control strategy (P, PI, PID, lead-lag).
- **Analysis**: Verify stability and performance using time-domain and frequency-domain methods.
- **Implementation**: Realize the controller physically or digitally.

2. The Gain and Phase Margin

These are stability margins that measure how far a system is from instability.

Gain Margin (GM):

- The amount of gain increase required to make the system marginally stable.
- Measured in dB at the phase crossover frequency (where phase = -180°).
- **Formula**: GM = $20 \log_{10}(1/|G(j\omega_p)|)$, where ω_p is the phase crossover frequency.
- **Acceptable value**: Typically > 6 dB.

Phase Margin (PM):

- The amount of phase shift required to make the system marginally stable.
- Measured in degrees at the gain crossover frequency (where $|G(j\omega)| = 1$).
- **Formula**: PM = $180° + \angle G(j\omega g)$, where ωg is the gain crossover frequency.
- **Acceptable value**: Typically > 45°.

```
Copy
Nyquist Plot Stability Margins:
    Im
```

```
↑
|    * (Gain crossover point, |G|=1)
|   /
|  /    PM = angle to -180° line
| /_____→ Re
|/
* (Phase crossover point, phase=-180°)
```

3. Linear Systems and Their Description

Time-Domain:

- **Differential Equations**: Described by linear ODEs with constant coefficients.
- **State-Space**: $\dot{x} = Ax + Bu$, $y = Cx + Du$
- **Impulse Response**: $h(t) = L^{-1}\{H(s)\}$
- **Step Response**: Shows transient behavior, overshoot, settling time.

Frequency-Domain:

- **Transfer Function**: $H(s) = Y(s)/U(s) = (b_m s^m + ... + b_0)/(s^n + a_{n-1}s^{n-1} + ... + a_0)$
- **Bode Plot**: Magnitude (dB) and phase (°) vs. frequency.
- **Nyquist Plot**: Real vs. imaginary parts of $G(j\omega)$.
- **Poles and Zeros**: Determine stability (poles must be in left-half plane).

4. Signal Transfer in Control Systems

Signal transfer describes how signals propagate through the system components.

Open-Loop Transfer:

- Signal flows only from input to output without feedback.
- **Formula**: $Y(s) = G(s)U(s)$

Closed-Loop Transfer:

- Signal returns through feedback path.
- **Formula**: $Y(s) = G(s)U(s)/(1 + G(s)H(s))$ for unity feedback

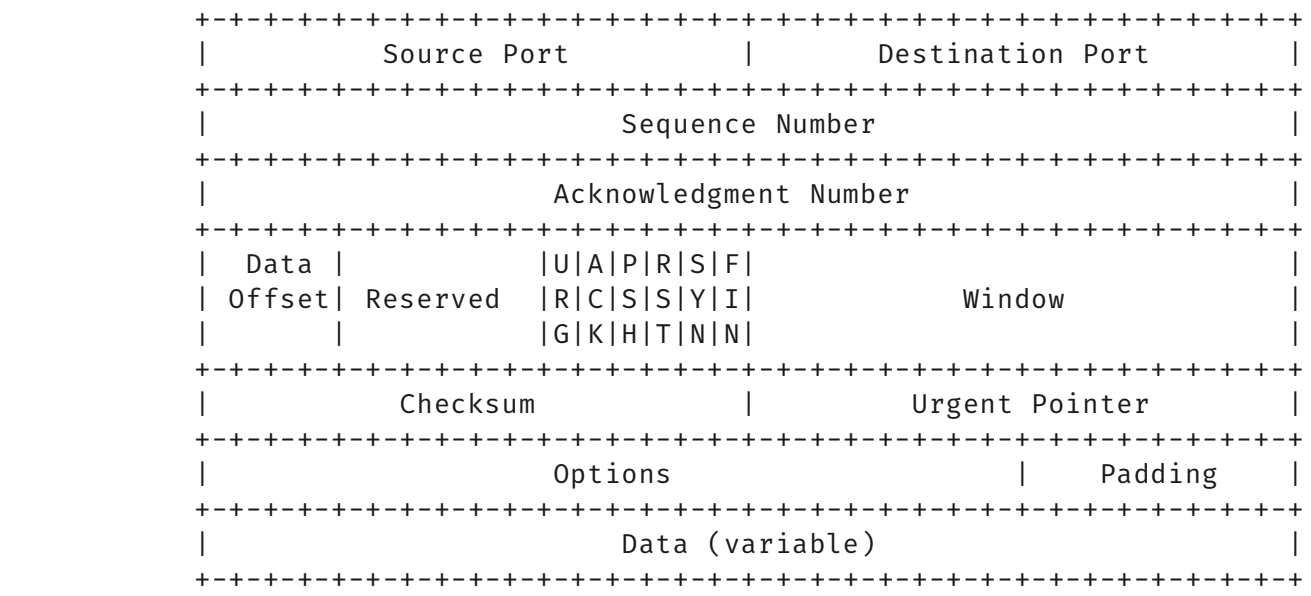
Signal Flow Graph:

```
Copy
   R(s) →[G(s)]→ Y(s)
            ↑      |
            |      |
            └[H(s)]┘
```

5. TCP and UDP Protocol Data Units (PDU)

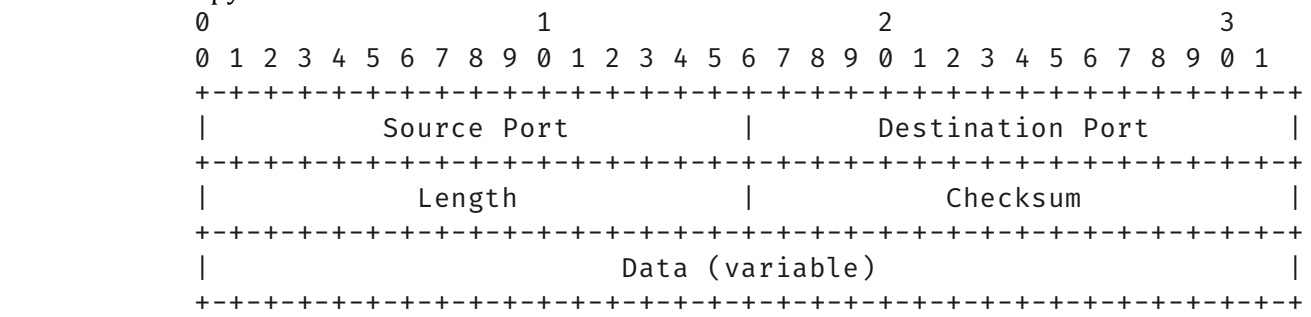
TCP Segment PDU:

```
Copy
  0                   1                   2                   3
  0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
```



UDP Datagram PDU:

Copy



6. Characteristics and Differences

Table

Copy

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable (acknowledgments, retransmission)	Unreliable
Ordering	Ordered delivery	No ordering guarantee
Flow Control	Yes (sliding window)	No
Congestion Control	Yes (AIMD, slow start)	No
Header Size	20-60 bytes	8 bytes

Feature	TCP	UDP
Speed	Slower (overhead)	Faster (minimal overhead)
Use Cases	Web browsing, email, file transfer	Streaming, gaming, DNS

Topic 3: Combinational logic design. Multiplexers/Demultiplexers. Encoders/Decoders. Comparators. Parity generators/checkers. Arithmetical logical units. Problem solving with search, uninformed and heuristic search algorithms. Constraint satisfaction problems.

1. Combinational Logic Design

Combinational logic circuits produce outputs based solely on current inputs, with no memory. They are built from basic gates (AND, OR, NOT, XOR).

Design Steps:

1. **Specification**: Define inputs and outputs.
2. **Truth Table**: List all input combinations and desired outputs.
3. **Simplification**: Use Boolean algebra or Karnaugh maps.
4. **Implementation**: Realize with gates.

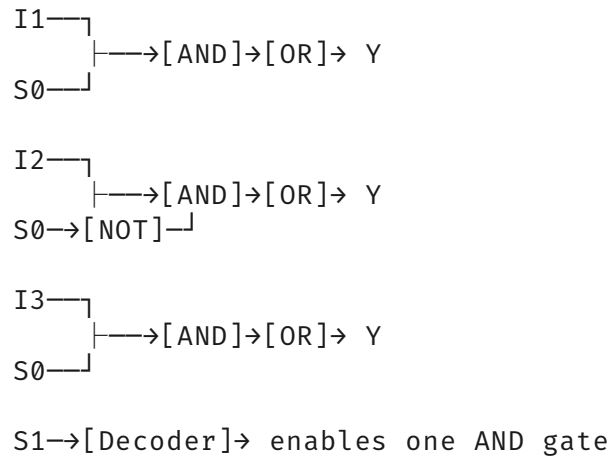
2. Multiplexers/Demultiplexers

Multiplexer (MUX):

- Selects one of many inputs and forwards it to a single output.
- **2^n inputs → 1 output** with n select lines.
- **4:1 MUX Truth Table:**

```
Copy
Select (S1 S0) | Output
---------------|-------
0 0            | I0
0 1            | I1
1 0            | I2
1 1            | I3
```

```
Copy
4:1 MUX Logic Diagram:
     I0──┐
         ├──→[AND]→[OR]→ Y
     S0─→[NOT]─┘
```



Demultiplexer (DEMUX):

- Routes one input to one of many outputs.
- **1 input → 2^n outputs** with n select lines.

3. Encoders/Decoders

Encoder:

- Converts 2^n inputs to n outputs (priority encoder resolves ties).
- **4:2 Encoder:** 0001→00, 0010→01, 0100→10, 1000→11

Decoder:

- Converts n inputs to 2^n outputs (one-hot).
- **2:4 Decoder:**

```
Copy
A B | Y0 Y1 Y2 Y3
----|------------
0 0 | 1  0  0  0
0 1 | 0  1  0  0
1 0 | 0  0  1  0
1 1 | 0  0  0  1
```

4. Comparators

Compares two binary numbers and indicates their relationship.

- **1-bit Comparator:**

```
Copy
A B | A>B  A=B  A<B
----|---------------
0 0 | 0    1    0
0 1 | 0    0    1
1 0 | 1    0    0
1 1 | 0    1    0
```

n-bit Comparator: Cascade 1-bit comparators for MSB to LSB.

5. Parity Generators/Checkers

Parity Generator:

- Adds a parity bit to make total number of 1s either even or odd.
- **Even Parity:** P = XOR of all data bits

Parity Checker:

- Validates received data by checking parity bit.
- **Error = XOR of all bits (including parity)**

```
Copy
4-bit Even Parity Generator:
Data: D3 D2 D1 D0
P = D3 ⊕ D2 ⊕ D1 ⊕ D0
Transmitted: D3 D2 D1 D0 P
```

6. Arithmetic Logic Unit (ALU)

Performs arithmetic and logical operations.

- **Operations**: ADD, SUB, AND, OR, XOR, NOT, SHIFT
- **Control Signals**: Select operation (e.g., 3-bit S2S1S0 selects 8 ops)
- **Structure:**

```
Copy
          A (n-bits) →┐
                      ├──→[ Operation ]→ Result (n-bits)
          B (n-bits) →┤       Decoder
    Control (k-bits) →┘
```

7. Problem Solving with Search

Search Space: Set of all possible states. **Goal**: Find path from initial to goal state.

Uninformed Search:

- **Breadth-First Search (BFS)**: Expands shallowest nodes first.
- **Depth-First Search (DFS)**: Expands deepest nodes first.
- **Uniform-Cost Search**: Expands cheapest path first.

Heuristic Search:

- **A* Algorithm**: f(n) = g(n) + h(n) where g(n) = cost so far, h(n) = heuristic estimate
- **Greedy Best-First**: Expands node with lowest h(n)

```
Copy
BFS vs DFS Tree:
      A
     / \
    B   C
```

```
   / \   \
  D   E   F

BFS order: A → B → C → D → E → F
DFS order: A → B → D → E → C → F
```

8. Constraint Satisfaction Problems (CSP)

Definition: Problems defined by variables, domains, and constraints.

- **Variables**: X = {X1, X2, ..., Xn}
- **Domains**: D = {D1, D2, ..., Dn}
- **Constraints**: C = {C1, C2, ..., Cm}

Solving Methods:

- **Backtracking**: Assign variables recursively, backtrack on failure.
- **Forward Checking**: Reduce domains of unassigned variables.
- **Constraint Propagation**: Enforce local consistency (arc consistency).
- **Heuristics**: Minimum remaining values (MRV), degree heuristic.

Example - Map Coloring:

```
Copy
Variables: WA, NT, SA, Q, NSW, V, T
Domains: {Red, Green, Blue}
Constraints: Adjacent regions ≠ color
```

Topic 4: The SSH protocol, key generation, SSH configuration settings. The principles of control, feedback control and open loop control. Set point control and reference signal tracking, the role of negative feedback. Requirements for control systems.

1. SSH Protocol

SSH (Secure Shell) is a cryptographic network protocol for secure remote access.

Key Components:

- **Transport Layer**: Server authentication, encryption, integrity
- **Authentication Layer**: Client authentication
- **Connection Layer**: Multiplexed channels

How it Works:

1. **TCP Connection**: Client connects to server port 22
2. **Version Exchange**: Protocol version negotiation
3. **Key Exchange**: Diffie-Hellman to establish shared secret
4. **Server Authentication**: Verified via host key

5. **Client Authentication**: Password, public key, or other methods
6. **Encrypted Session**: Data flows through secure channel

2. Key Generation

SSH Key Types:

- **RSA**: Most common, 2048-4096 bits
- **DSA**: Deprecated, 1024 bits
- **ECDSA**: Elliptic curve, faster
- **Ed25519**: Modern, secure, fast

Key Generation Process:

bash

```
Copy
# Generate RSA key pair
ssh-keygen -t rsa -b 4096 -C "user@host"

# Files created:
~/.ssh/id_rsa (private key - protect with 600 permissions)
~/.ssh/id_rsa.pub (public key - can be shared)
```

Public Key Format:

```
Copy
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC... user@host
[algorithm] [base64-key] [comment]
```

3. SSH Configuration Settings

Client Config (`~/.ssh/config`):

```
Copy
Host server1
    HostName 192.168.1.100
    User admin
    Port 2222
    IdentityFile ~/.ssh/id_server1
    Compression yes
    ForwardAgent yes
```

Server Config (`/etc/ssh/sshd_config`):

```
Copy
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
AllowUsers user1 user2
```

4. Principles of Control

Open-Loop Control:

- No feedback path
- Output has no effect on control action
- **Formula**: Controller → Plant → Output
- **Disadvantage**: Cannot correct disturbances

Closed-Loop (Feedback) Control:

- Output measured and fed back
- Error signal drives controller
- **Formula**: Error = Reference - Feedback

```
Copy
Open-Loop:
r → [Controller] → u → [Plant] → y

Closed-Loop:
      ┌──────────────────┐
      ↓                  |
r → [⊕] → e → [Controller] → u → [Plant] → y
      ↑                  |
      └──[Sensor]────────┘
```

5. Set Point Control and Reference Tracking

Set Point Control: Maintains output at desired constant value (set point). **Reference Tracking**: Output follows changing reference signal.

Negative Feedback Role:

- Reduces error between reference and output
- Improves stability
- Reduces sensitivity to parameter variations
- Rejects disturbances

```
Copy
Negative Feedback Effect:
Without feedback: 10% plant variation → 10% output variation
With feedback: 10% plant variation → 0.1% output variation
```

6. Requirements for Control Systems

- **Stability**: System must be stable (poles in LHP)
- **Performance**: Meet transient specs (overshoot < 10%, settling time < 5s)
- **Accuracy**: Zero steady-state error
- **Robustness**: Tolerate model uncertainties

- **Disturbance Rejection**: Minimize external disturbance effects
- **Noise Attenuation**: Reject measurement noise

Topic 5: Two-person games and their representations. Winning strategy, optimal decisions in games. MOS transistor: large signal model and characteristics. The MOS transistor as a switch. CMOS inverter, basic logic gates. The operational amplifier. Negative feedback. Basic applications.

1. Two-Person Games Representations

Normal (Strategic) Form:

- Players, strategies, payoff matrix

- **Example (Prisoner's Dilemma):**

```
Copy
                Player 2
                Cooperate  Defect
Player 1 Cooperate  (-1,-1)  (-3,0)
         Defect     (0,-3)   (-2,-2)
```

Extensive Form:

- Game tree with nodes (decision points) and edges (actions)

- Information sets show what players know

2. Winning Strategy and Optimal Decisions

Winning Strategy: A strategy that guarantees victory regardless of opponent's moves.

- **Deterministic Games**: Perfect information (chess, checkers)
- **Game Tree Search**: Minimax algorithm

Minimax Algorithm:

```
Copy
function minimax(node, depth, maximizing):
    if depth == 0 or terminal(node):
        return evaluate(node)

    if maximizing:
        value = -∞
        for child in node.children:
            value = max(value, minimax(child, depth-1, false))
        return value
    else:
        value = +∞
        for child in node.children:
```

```
            value = min(value, minimax(child, depth-1, true))
        return value
```

3. MOS Transistor Large-Signal Model

Regions of Operation:

Cutoff Region (Vgs < Vth):

- No channel, no current
- Id = 0

Triode (Linear) Region (Vgs > Vth, Vds < Vgs - Vth):

- Channel continuous from source to drain
- $Id = \mu_n C_{ox}(W/L)[(Vgs-Vth)Vds - Vds^2/2]$

Saturation Region (Vgs > Vth, Vds ≥ Vgs - Vth):

- Channel pinches off at drain
- $Id = (\mu_n C_{ox}/2)(W/L)(Vgs - Vth)^2(1 + \lambda Vds)$

```
Copy
MOS Characteristic Curves:
Id ↑
   |    Vgs increasing
   |    / / / /
   |   / / / /
   |  / / / /  ← Saturation region
   | / / / /
   |/ / / /___  ← Triode region
   └─────────→ Vds
```

4. MOS Transistor as a Switch

Digital Operation:

- **OFF**: Vgs = 0 < Vth → Cutoff → No current → Open switch
- **ON**: Vgs = Vdd > Vth → Triode → Low resistance → Closed switch

Equivalent Circuit:

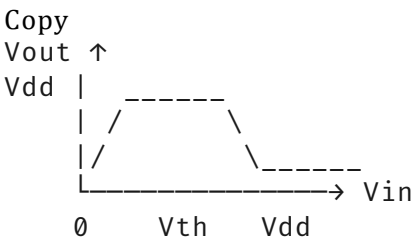
```
Copy
OFF State:
Source ———[∞Ω]——— Drain

ON State:
Source ———[Ron]——— Drain  (Ron ≈ few kΩ to few hundred Ω)
```

5. CMOS Inverter

Structure: PMOS (top) + NMOS (bottom)

- **Input Low (0V)**: PMOS ON, NMOS OFF → Output High (Vdd)
- **Input High (Vdd)**: PMOS OFF, NMOS ON → Output Low (0V)

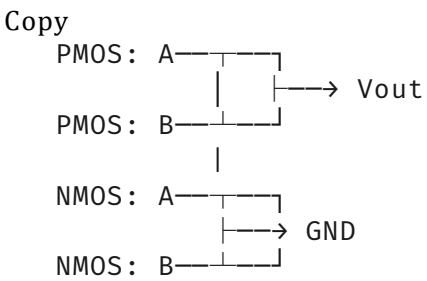
Voltage Transfer Characteristic:



Power Consumption: Only during switching (dynamic power)

6. Basic Logic Gates (CMOS)

NAND Gate:

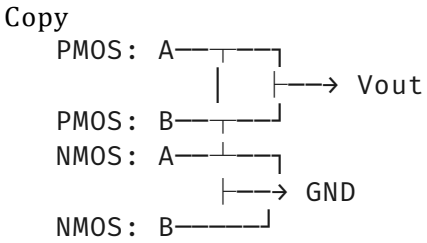


Truth Table:

Copy

A	B	Out
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate:



7. Operational Amplifier

Ideal Op-Amp Characteristics:

- Infinite open-loop gain (Aol → ∞)
- Infinite input impedance (Rin → ∞)
- Zero output impedance (Rout → 0)
- Infinite bandwidth
- Virtual short: V+ = V- (when negative feedback)

Practical Op-Amp (741):

- Aol ≈ 100 dB (100,000)
- Rin ≈ 2 MΩ
- Rout ≈ 75 Ω
- GBW ≈ 1 MHz

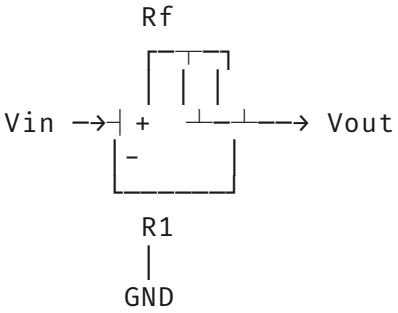
8. Negative Feedback

Principle: Output fed back to inverting input. **Effects:**

- Reduces gain: Af = A/(1 + Aβ)
- Increases bandwidth
- Reduces distortion
- Stabilizes operation

Non-Inverting Amplifier:

Copy




```
Vout = Vin × (1 + Rf/R1)
```

9. Basic Applications

- **Inverting Amplifier**: Vout = -Vin × (Rf/Rin)
- **Voltage Follower**: Vout = Vin (buffer)
- **Summing Amplifier**: Vout = -Σ(Vi × Rf/Ri)
- **Difference Amplifier**: Vout = (V2 - V1) × (Rf/Rin)
- **Integrator**: Vout = -1/RC ∫Vin dt
- **Differentiator**: Vout = -RC dVin/dt

Topic 6: Sequential logical: Latches and Flip-Flops. Counters. Shift registers. Memories. New elements of HTML5. New features of CSS3.

Control structures in web scripts. Sensor through a web page. Providing remote management systems through a web page.

1. Latches (Level-Sensitive)

SR Latch (NOR implementation):

```
Copy
S ──┬──→ Q
    │
    └─→[NOR]──┬──→ Q'
              │
R ────────────┴──→[NOR]──→ Q

Q' = NOT(R OR Q)
Q = NOT(S OR Q')
```

Truth Table:

```
Copy
S R | Q  Q' | State
0 0 | Q  Q' | Hold
0 1 | 0  1  | Reset
1 0 | 1  0  | Set
1 1 | 0  0  | Invalid
```

D Latch:

```
Copy
Enable ──┬──→[AND]──→ Q
         │
D ───────┴──→[AND]──→ Q'

Q follows D when Enable=1, holds when Enable=0
```

2. Flip-Flops (Edge-Triggered)

D Flip-Flop:

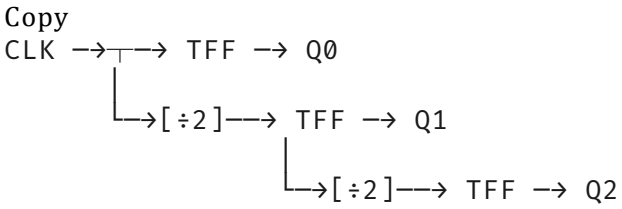
```
Copy
CLK ─→┬──→ Master D Latch ─→ Slave D Latch ─→ Q
      │     (CLK=1)           (CLK=0)
      │
D ────┴──────────────────────────────→

Q updates only on rising edge of CLK
```

JK Flip-Flop:

```
Copy
J K | Q_next | Function
0 0 | Q      | Hold
0 1 | 0      | Reset
1 0 | 1      | Set
1 1 | Q'     | Toggle
```

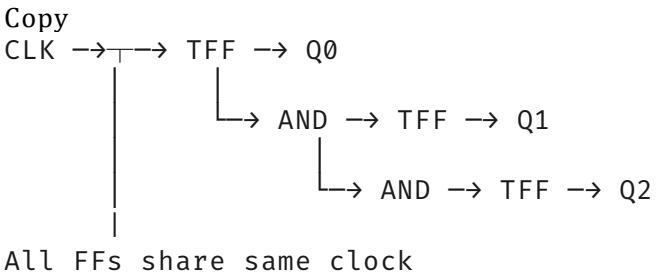
3. Counters

Asynchronous (Ripple) Counter:



Each flip-flop clocks the next one → propagation delay accumulates

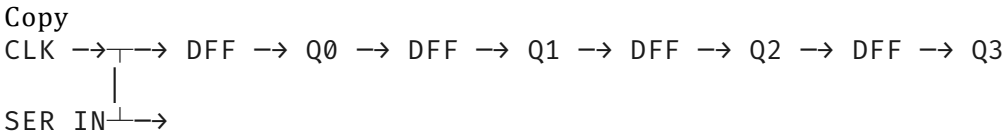
Synchronous Counter:



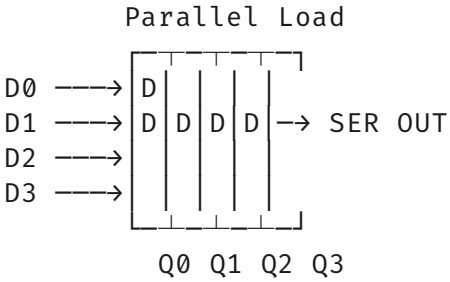
Mod-n Counter: Counts 0 to n-1, then resets

4. Shift Registers

Serial-In Parallel-Out (SIPO):



Parallel-In Serial-Out (PISO):



Applications: Serial-parallel conversion, delay lines, multiplication by 2

5. Memories

SRAM (Static RAM):

- Uses flip-flops to store each bit
- Fast (access time ~10ns)
- Volatile

- 6 transistors per bit
- Used for cache

DRAM (Dynamic RAM):

- Uses capacitor to store charge
- Slower, needs refresh (every few ms)
- Volatile
- 1 transistor + 1 capacitor per bit
- Used for main memory

Memory Organization:

Copy

```
Address —→[Decoder]—→ Word Line —→[Memory Array]—→ Bit Lines
                                                        ↓
                                                [Sense Amplifiers]—→ Data
Out
```

6. New Elements of HTML5

Semantic Elements:

HTML

Preview

Copy

```html
<header>     <!-- Page or section header -->
<nav>        <!-- Navigation links -->
<main>       <!-- Main content -->
<article>    <!-- Independent content -->
<section>    <!-- Thematic grouping -->
<aside>      <!-- Sidebar content -->
<footer>     <!-- Footer information -->
<figure>     <!-- Self-contained content -->
<figcaption> <!-- Caption for figure -->
```

Form Elements:

HTML

Preview

Copy

```html
<input type="email">       <!-- Validates email format -->
<input type="date">        <!-- Date picker -->
<input type="range">       <!-- Slider control -->
<input type="color">       <!-- Color picker -->
<datalist>                  <!-- Auto-complete list -->
```

Media Elements:

HTML

Preview

Copy
```
<video controls>              <!-- Native video player -->
<audio controls>              <!-- Native audio player -->
<canvas>                        <!-- 2D/3D graphics -->
<svg>                           <!-- Vector graphics -->
```

7. New Features of CSS3

Selectors:

css

Copy
```
/* Attribute starts with */
a[href^="https"] { color: green; }

/* nth-child */
li:nth-child(odd) { background: #eee; }

/* Pseudo-elements */
p::first-letter { font-size: 2em; }
```

Layout:

css

Copy
```
/* Flexbox */
.container {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Grid */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

Visual Effects:

css

Copy
```
/* Transitions */
.box {
  transition: transform 0.3s ease;
}

/* Animations */
@keyframes slide {
  from { left: 0; }
  to { left: 100px; }
}

/* Box Shadow */
.card {
```

```
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

/* Border Radius */
button {
  border-radius: 8px;
}
```

8. Control Structures in Web Scripts

JavaScript Control Flow:

JavaScript

Copy

```
// Conditional
if (temperature > 30) {
  console.log("Hot");
} else if (temp > 20) {
  console.log("Warm");
} else {
  console.log("Cool");
}

// Loop
for (let i = 0; i < 10; i++) {
  console.log(i);
}

// While
while (sensor.active) {
  readData();
}

// Switch
switch(device.status) {
  case 'online': updateUI(); break;
  case 'offline': showError(); break;
  default: checkConnection();
}
```

9. Sensor Through a Web Page

Implementation using ESP32/Arduino and WebSocket:

JavaScript

Copy

```
// Server-side (Node.js + Socket.io)
const sensor = require('node-dht-sensor');
const io = require('socket.io')(server);

setInterval(() => {
  sensor.read(22, 4, (err, temp, hum) => {
    io.emit('sensorData', { temperature: temp, humidity: hum });
  });
}, 2000);
```

```
// Client-side HTML
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();
  socket.on('sensorData', (data) => {
    document.getElementById('temp').textContent = data.temperature;
    document.getElementById('hum').textContent = data.humidity;
  });
</script>
<body>
  <div>Temperature: <span id="temp">--</span>°C</div>
  <div>Humidity: <span id="hum">--</span>%</div>
</body>
```

10. Remote Management Systems Through Web Page

Architecture:

```
Copy
Browser → HTTPS → Web Server → API → Device Manager → Devices
          (nginx)   (Node.js)    (MQTT/SSH)
```

Features:

- **Dashboard**: Real-time status overview
- **Configuration**: Update device settings via forms
- **Control**: Send commands (GPIO control)
- **Monitoring**: Charts with historical data (Chart.js)
- **Authentication**: JWT tokens, session management
- **Authorization**: Role-based access (admin, user)

Security Considerations:

- Use HTTPS with TLS 1.3
- Implement CSRF tokens
- Sanitize all inputs
- Use WebSocket Secure (WSS)
- Implement rate limiting

Topic 7: Basics of software and hardware testing, basic testing methodologies, test levels, unit testing through examples from advanced programming languages and / or hardware description languages. Implementation of control structures in assembly language (conditional

and unconditional control transfer, branching, cycle organization, subroutine call).

1. Software Testing Basics

Testing Definition: Process of executing a program with intent to find errors.

Testing Objectives:

- Verify requirements are met
- Find defects
- Validate functionality
- Ensure reliability

Testing Principles:

- Early testing saves cost
- Exhaustive testing is impossible
- Defect clustering (80% defects in 20% code)
- Pesticide paradox (need new test cases)

2. Testing Methodologies

Black-Box Testing:

- Tests functionality without knowing internal structure
- Based on specifications
- Techniques: Equivalence partitioning, boundary value analysis, decision tables

White-Box Testing:

- Tests internal structure and logic
- Requires code knowledge
- Techniques: Statement coverage, branch coverage, path coverage

Gray-Box Testing:

- Combination of both approaches
- Partial knowledge of internals

3. Test Levels

Unit Testing:

- Individual components/modules
- Done by developers
- **Example (Python):**

Python

Copy

```python
import unittest

class TestAdder(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(adder(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(adder(-2, -3), -5)

if __name__ == '__main__':
    unittest.main()
```

Integration Testing:

- Multiple units together
- Top-down, bottom-up, sandwich approaches

System Testing:

- Complete integrated system
- Functional and non-functional testing

Acceptance Testing:

- User verification
- Alpha, beta, UAT (User Acceptance Testing)

4. Unit Testing in HDL (VHDL Example)

vhdl

Copy

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity test_adder is
end entity;

architecture tb of test_adder is
    component adder
        port(a, b: in std_logic_vector(3 downto 0);
             sum: out std_logic_vector(3 downto 0));
    end component;

    signal a, b, sum: std_logic_vector(3 downto 0);
```

```
begin
    UUT: adder port map(a, b, sum);

    process
    begin
        a <= "0010"; b <= "0011"; wait for 10 ns;
        assert sum = "0101" report "Test 1 failed" severity error;

        a <= "1111"; b <= "0001"; wait for 10 ns;
        assert sum = "0000" report "Test 2 failed" severity error;

        wait;
    end process;
end architecture;
```

5. Assembly Language Control Structures

Conditional Control Transfer:

assembly

Copy

```
; x86 Assembly - Compare and Jump
mov eax, [num1]
mov ebx, [num2]
cmp eax, ebx        ; Compare EAX and EBX
je  equal_label     ; Jump if Equal
jg  greater_label   ; Jump if Greater (signed)
jl  less_label      ; Jump if Less (signed)
jne not_equal_label ; Jump if Not Equal

equal_label:
    ; Code for equal case
    jmp continue

greater_label:
    ; Code for greater case
    jmp continue
```

Unconditional Control Transfer:

assembly

Copy

```
jmp target_label    ; Always jump
call subroutine     ; Jump to subroutine, save return address
ret                 ; Return from subroutine

target_label:
    ; Continue execution
```

6. Branching Structures

If-Then-Else:

assembly

Copy

```
; If (ax > 10) then bx = 1 else bx = 0
cmp ax, 10
jle else_block
    mov bx, 1       ; Then block
    jmp end_if
else_block:
    mov bx, 0       ; Else block
end_if:
```

Switch-Case:

assembly

Copy

```
; Switch on AL register
cmp al, 1
je case1
cmp al, 2
je case2
cmp al, 3
je case3
jmp default_case

case1:
    ; Handle case 1
    jmp switch_end
case2:
    ; Handle case 2
    jmp switch_end
switch_end:
```

7. Loop Organization

For Loop:

assembly

Copy

```
; for (ecx = 10; ecx > 0; ecx--) { ... }
mov ecx, 10         ; Initialize counter
loop_start:
    ; Loop body
    dec ecx         ; Decrement
    jnz loop_start  ; Jump if Not Zero
```

While Loop:

assembly

Copy

```
; while (eax < 100) { ... }
mov eax, 0
while_start:
    cmp eax, 100
    jge while_end
    ; Loop body
    add eax, 1
    jmp while_start
```

```
while_end:
```

Do-While Loop:**

assembly

```
Copy
; do { ... } while (eax < 100);
do_start:
    ; Loop body
    add eax, 1
    cmp eax, 100
    jl do_start
```

8. Subroutine Call

Structure:

assembly

```
Copy
main:
    push 5          ; Push argument
    push 10
    call add_numbers
    add esp, 8      ; Clean stack (cdecl convention)
    ; Result in eax

add_numbers:
    push ebp        ; Save base pointer
    mov ebp, esp    ; Set new stack frame
    mov eax, [ebp+8] ; Get first parameter
    mov ebx, [ebp+12]; Get second parameter
    add eax, ebx    ; Add them
    pop ebp         ; Restore base pointer
    ret             ; Return to caller
```

Calling Conventions:

- **cdecl**: Caller cleans stack, parameters right-to-left
- **stdcall**: Callee cleans stack (Windows API)
- **fastcall**: First 2 params in registers (ECX, EDX)

Topic 8: Typical peripherals and communication protocols for embedded systems. Describe the control unit, peripherals, and applicability of a single-board mini-computer in embedded systems. Describe the functions and services of network management systems and the possibilities of implementing these functions for specific products (for example MRTG or Nagios).

1. Typical Embedded Peripherals

Sensors:

- **Temperature**: DS18B20 (1-Wire), LM35 (Analog)
- **Humidity**: DHT22 (Digital)
- **Motion**: PIR sensors
- **Distance**: Ultrasonic (HC-SR04), IR sensors
- **Accelerometer**: MPU6050 (I2C)

Actuators:

- **Motors**: DC, Stepper, Servo
- **Relays**: Electromechanical, Solid-state
- **LEDs**: Simple, RGB (PWM control)
- **Displays**: LCD (1602), OLED (I2C), TFT (SPI)

Storage:

- **EEPROM**: I2C, small data retention
- **SD Card**: SPI, large storage
- **Flash**: On-chip (program memory)

2. Communication Protocols

I2C (Inter-Integrated Circuit):

- 2-wire: SDA (data), SCL (clock)
- Master-slave, multi-master capable
- 7-bit or 10-bit addressing
- Speed: Standard (100 kHz), Fast (400 kHz), High-speed (3.4 MHz)
- **Pull-up resistors required**

```
Copy
I2C Timing:
SCL: ‾|_|‾|_|‾|_|‾|_|‾‾
     |___|   |___|   |___|

SDA: ‾‾‾|_______|‾‾‾‾‾‾‾|_______
       Start    Data    Stop
```

SPI (Serial Peripheral Interface):

- 4-wire: MOSI, MISO, SCK, CS
- Full duplex, master-slave
- Speed: Up to tens of MHz
- **No addressing, CS selects slave**

```
Copy
SPI Timing (Mode 0):
SCK: ─┐   ┌───┐   ┌───┐   ┌──
     └───┘   └───┘   └───┘

MOSI: ────┐       ┌───────┐
          └───────┘       └────

MISO: ────┐       ┌───────┐
          └───────┘       └────
          CS=0    Data transfer
```

UART (Universal Asynchronous Receiver-Transmitter):

- 2-wire: TX, RX (plus GND)
- Asynchronous, no clock
- Baud rate must match
- Frame: Start bit + Data (5-9 bits) + Parity + Stop bit(s)

```
Copy
UART Frame:
Start │ Data (LSB first) │ Parity │ Stop
  0   │   1 0 1 1 0 0 1  │   1    │  1
      └──────────────────┴────────┘

      1 start, 8 data, 1 parity, 1 stop = 11 bits
```

1-Wire:

- Single data line (plus ground)
- Dallas Semiconductor protocol
- Each device has unique 64-bit ROM ID
- Master initiates all communication

3. Single-Board Mini-Computer (e.g., Raspberry Pi Pico)

Control Unit:

- **RP2040 MCU**: Dual-core ARM Cortex-M0+ @ 133 MHz
- **Memory**: 264 KB SRAM, external QSPI Flash
- **GPIO**: 26 multi-function pins
- **Peripherals**: 2 UART, 2 SPI, 2 I2C, 16 PWM, USB 1.1

Peripherals Integration:

```
Copy
Pico GPIO Map:
GP0  → UART0_TX / I2C0_SDA / SPI0_RX
GP1  → UART0_RX / I2C0_SCL / SPI0_CSn
GP2  → UART0_CTS / I2C1_SDA / SPI0_SCK
GP3  → UART0_RTS / I2C1_SCL / SPI0_TX
GP4  → UART1_TX / I2C0_SDA / SPI0_RX
```

```
...
GP26 → ADC0 / I2C1_SDA / SPI1_SCK
GP27 → ADC1 / I2C1_SCL / SPI1_TX
GP28 → ADC2 / I2C0_SDA / SPI1_RX
```

Applicability in Embedded Systems:

- **Prototyping**: Quick development and testing
- **Education**: Learning embedded programming
- **Low-Cost Automation**: Home automation, small robots
- **IoT**: Sensor nodes, data loggers
- **Limitations**: No OS, limited memory, single-core effective

4. Network Management Systems (NMS) Functions

FCAPS Model:

- **F**ault Management: Detect, isolate, correct faults
- **C**onfiguration Management: Track device configs
- **A**ccounting Management: Resource usage tracking
- **P**erformance Management: Monitor and optimize
- **S**ecurity Management: Access control, encryption

Key Services:

- **Discovery**: Auto-detect network devices
- **Monitoring**: Real-time status and metrics
- **Alerting**: Notifications via email/SMS
- **Reporting**: Historical analysis and trends
- **Configuration Backup**: Store and restore configs
- **Event Correlation**: Link related events

5. MRTG (Multi Router Traffic Grapher)

Functions:

- Monitors traffic load on network links
- Creates HTML pages with PNG graphs
- Uses SNMP to collect data
- Stores data in Round-Robin Database (RRD)

Implementation:

bash

Copy

```
# Installation
sudo apt-get install mrtg
```

```
# Configuration (mrtg.cfg)
Target[router1_eth0]: 2:public@192.168.1.1
MaxBytes[router1_eth0]: 1250000
Title[router1_eth0]: Router 1 Ethernet 0
PageTop[router1_eth0]: <h1>Traffic Analysis</h1>

# Cron job (every 5 minutes)
*/5 * * * * /usr/bin/mrtg /etc/mrtg.cfg
```

Graph Output:

- Daily, weekly, monthly, yearly views
- Shows inbound/outbound traffic
- Auto-scaling Y-axis

6. Nagios

Functions:

- Host and service monitoring
- Alert escalation
- Event handling
- GUI dashboard

Architecture:

```
Copy
Nagios Core
     │
     ├──→ Scheduler → Check workers
     │                    ↓
     ├──← Results ← Plugins (SNMP, NRPE, etc.)
     │
     ├──→ Notification system
     │
     └──→ Web UI (CGI)
```

Configuration:

cfg

```
Copy
# Host definition
define host {
    host_name           web-server
    alias               Main Web Server
    address             192.168.1.10
    check_command       check-host-alive
    max_check_attempts  3
}
```

```
# Service definition
define service {
    host_name           web-server
    service_description HTTP
    check_command       check_http
    check_interval      5
}
```

Plugins:

- **check_ping**: Device reachability
- **check_http**: Web server response
- **check_snmp**: Interface status, CPU, memory
- **check_nrpe**: Remote script execution

Notification Example:

```
Copy
Service CRITICAL: HTTP on web-server
State: CRITICAL
Info: Connection refused
When: 2025-12-29 14:30:00
Action: Escalate to on-call engineer
```

Topic 9: Programmable logic devices. Designing a digital system in hardware description language, and implementing it in FPGA devices. Basic concepts of system engineering, different paradigms; Characteristics of classical methodologies, waterfall (structured) model; Iterative (evolutionary, incremental) models; Agile software development methodologies, tools; OO design principles and important design patterns, MVC architectural pattern.

1. Programmable Logic Devices (PLD) Hierarchy

SPLD (Simple PLD):

- **PLA (Programmable Logic Array)**: AND array programmable, OR array programmable
- **PAL (Programmable Array Logic)**: AND array programmable, OR array fixed
- **GAL (Generic Array Logic)**: Reprogrammable PAL

CPLD (Complex PLD):

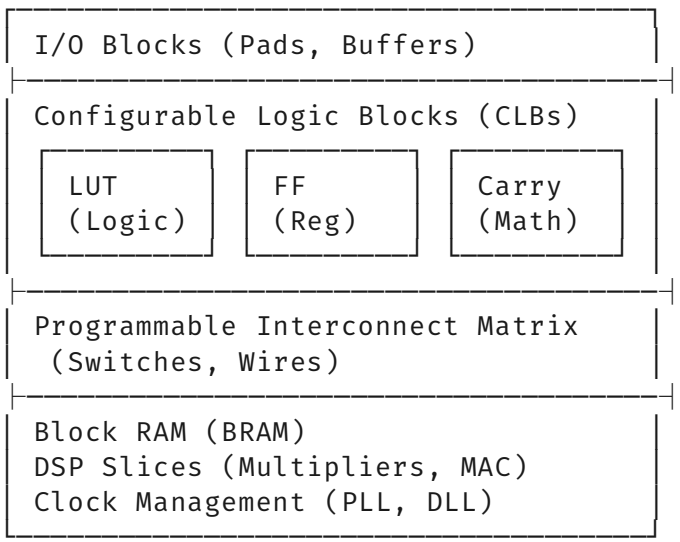
- Multiple PAL-like blocks (macrocells)
- Interconnected via programmable switch matrix
- Examples: Xilinx XC9500, Altera MAX7000

FPGA (Field-Programmable Gate Array):

- Matrix of Configurable Logic Blocks (CLBs)
- Programmable interconnects
- Block RAM, DSP slices, I/O blocks
- Examples: Xilinx Artix-7, Altera Cyclone

2. FPGA Architecture

```
Copy
FPGA Structure:
```



CLB Details:

- **LUT (Look-Up Table)**: 4-6 input SRAM implementing any logic function
- **Flip-Flop**: Configurable register
- **Carry Chain**: Fast arithmetic operations
- **Configuration**: SRAM cells define LUT content and routing

3. Hardware Description Language Design Flow

```
Copy
Design Entry (VHDL/Verilog)
    ↓
Simulation (ModelSim, GHDL)
    ↓
Synthesis (Vivado, Quartus)
    ↓
Implementation:
  - Translate (Synthesis → Netlist)
  - Map (Netlist → FPGA primitives)
  - Place & Route (Assign positions, connect)
    ↓
```

```
Timing Analysis
    ↓
Configuration File (.bit, .sof)
    ↓
Download to FPGA
```

4. VHDL Example (4-bit Adder)

vhdl

Copy

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder4bit is
    port(
        a, b: in std_logic_vector(3 downto 0);
        cin: in std_logic;
        sum: out std_logic_vector(3 downto 0);
        cout: out std_logic
    );
end entity;

architecture behavioral of adder4bit is
    signal temp: unsigned(4 downto 0);
begin
    temp <= unsigned('0' & a) + unsigned('0' & b) + unsigned'('0' &
cin);
    sum <= std_logic_vector(temp(3 downto 0));
    cout <= temp(4);
end architecture;
```

5. System Engineering Concepts

Definition: Interdisciplinary approach to design, implement, and manage complex systems over their life cycles.

Key Principles:

- **Holistic View**: System as whole, not just parts
- **Life Cycle**: Requirements → Design → Implementation → Test → Deploy → Maintain
- **Stakeholder Analysis**: Identify all involved parties
- **Trade-off Analysis**: Balance performance, cost, schedule, risk

System Attributes:

- Functionality
- Performance
- Reliability

- Maintainability
- Usability
- Cost-effectiveness

6. Development Paradigms

Plan-Driven (Waterfall):

- Sequential phases
- Heavy documentation
- Fixed requirements
- Good for stable, well-understood problems

Agile:

- Iterative and incremental
- Adaptive to change
- Customer collaboration
- Working software over documentation

Hybrid:

- Plan-driven for architecture
- Agile for implementation

7. Waterfall Model

```
Copy
Requirements → System Design → Implementation → Testing → Deployment →
Maintenance
     ↓              ↓              ↓            ↓          ↓
↓
  Spec doc     Architecture   Coding      Test plans  Installation
Bug fixes
               Design doc     Unit tests  Integration
                                           System tests

Characteristics:
- Sequential, no overlap
- Review at each phase end
- Changes difficult after phase completion
- Works well for small, clear projects
```

Advantages:

- Simple to understand

- Clear milestones
- Easy management

Disadvantages:

- Late testing
- Inflexible to changes
- High risk

8. Iterative/Evolutionary Models

Incremental Model:

```
Copy
Req1 → Design1 → Build1 → Test1 → Deliver1
Req2 → Design2 → Build2 → Test2 → Deliver2
Req3 → Design3 → Build3 → Test3 → Deliver3
```

Each increment adds functionality

Spiral Model:

```
Planning → Risk Analysis → Engineering → Evaluation → Next Spiral
```

Risk-driven, expensive, good for large projects

RUP (Rational Unified Process):

- 4 phases: Inception, Elaboration, Construction, Transition
- Iterative within phases

9. Agile Methodologies

Core Values (Agile Manifesto):

- Individuals and interactions over processes/tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Scrum:

```
Copy
Product Backlog → Sprint Planning → Sprint (1-4 weeks)
                                        ↓
                                  Daily Standup (15 min)
                                        ↓
                                  Sprint Review → Increment
                                        ↓
                                  Sprint Retrospective
```

Roles: Product Owner, Scrum Master, Development Team **Artifacts**: Product Backlog, Sprint Backlog, Increment

Kanban:

- Visual board (To Do, In Progress, Done)
- Work In Progress (WIP) limits
- Continuous flow, no sprints

Extreme Programming (XP):

- Pair programming
- Test-Driven Development (TDD)
- Continuous Integration
- Refactoring

10. Agile Tools

- **JIRA**: Sprint planning, backlog management
- **Trello**: Kanban boards
- **GitLab/GitHub**: CI/CD, issue tracking
- **Slack/Teams**: Communication
- **Jenkins**: Continuous integration
- **Selenium**: Automated testing

11. OO Design Principles (SOLID)

Single Responsibility: One class, one reason to change **O**pen/Closed: Open for extension, closed for modification **L**iskov Substitution: Subtypes replaceable by base types **I**nterface Segregation: Small, specific interfaces **D**ependency Inversion: Depend on abstractions, not concretions

12. Important Design Patterns

Creational:

- **Singleton**: One instance globally
- **Factory**: Create objects without specifying exact class
- **Builder**: Construct complex objects step-by-step

Structural:

- **Adapter**: Convert interface to another
- **Decorator**: Add responsibilities dynamically

- **Facade**: Simplify complex subsystem

Behavioral:

- **Observer**: One-to-many dependency
- **Strategy**: Encapsulate algorithms
- **Command**: Encapsulate requests as objects

13. MVC Architectural Pattern

Model-View-Controller:

```
Copy
User Action → Controller → Model → Update → View → Display
                ↑                                  ↓
                └──────────── Observer ────────────┘
```

Model: Data and business logic

- Manages data, state, rules
- Notifies observers of changes
- Independent of UI

View: Presentation layer

- Displays model data
- Sends user input to controller
- Passive (no business logic)

Controller: Intermediary

- Handles user input
- Updates model
- Selects appropriate view

Example (Web Application):

```
Copy
Model: User class (id, name, email), database operations
View: HTML templates (user_profile.html)
Controller: Route handlers (app.get('/user/:id', ...))
```

Advantages:

- Separation of concerns
- Reusability (multiple views for same model)
- Parallel development
- Testability

Disadvantages:

- Complexity for simple apps
- Steep learning curve

Topic 10: Web server configuration using SSL, the OpenSSL cryptographic library basic functions: authentication, encryption. The instruction set architecture (ISA) of Intel X86 processors (registers, addressing, instructions, memory architecture, interrupt system).

1. SSL/TLS Configuration

SSL Certificate Types:

- **Domain Validated (DV)**: Proves domain ownership
- **Organization Validated (OV)**: Proves organization identity
- **Extended Validation (EV)**: Highest level, green bar

Certificate Generation with OpenSSL:

bash

Copy

```
# Generate RSA private key (2048-bit)
openssl genrsa -out server.key 2048

# Generate Certificate Signing Request (CSR)
openssl req -new -key server.key -out server.csr
# Country, State, Organization, Common Name (domain)

# Generate self-signed certificate (valid 365 days)
openssl x509 -req -days 365 -in server.csr -signkey server.key -out
server.crt

# View certificate details
openssl x509 -in server.crt -text -noout
```

2. OpenSSL Functions

Authentication:

- **Digital Signatures**: Prove message origin and integrity

bash

Copy

```
# Sign file
openssl dgst -sha256 -sign private.key -out file.sig file.txt

# Verify signature
openssl dgst -sha256 -verify public.key -signature file.sig file.txt
```

Encryption:

- **Symmetric**: Same key for encryption/decryption (AES)

bash

Copy

```
# Encrypt
openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.bin

# Decrypt
openssl enc -d -aes-256-cbc -in encrypted.bin -out decrypted.txt
```

- **Asymmetric**: Public/private key pair (RSA)

bash

Copy

```
# Encrypt with public key
openssl rsautl -encrypt -inkey public.pem -pubin -in secret.txt -out
encrypted.dat

# Decrypt with private key
openssl rsautl -decrypt -inkey private.pem -in encrypted.dat -out
decrypted.txt
```

Hashing:

bash

Copy

```
# SHA-256 hash
openssl dgst -sha256 file.txt
```

3. Apache SSL Configuration

apache

Copy

```
<VirtualHost *:443>
    ServerName www.example.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/server.crt
    SSLCertificateKeyFile /etc/ssl/private/server.key
    SSLCertificateChainFile /etc/ssl/certs/ca-bundle.crt

    SSLProtocol TLSv1.2 TLSv1.3
    SSLCipherSuite HIGH:!aNULL:!MD5

    <Directory /var/www/html>
        Require all granted
    </Directory>
</VirtualHost>
```

4. x86 Processor Registers

General Purpose (32-bit):

Copy

```
EAX: Accumulator (arithmetic, I/O)
```

```
EBX: Base (pointer to data)
ECX: Counter (loops, shifts)
EDX: Data (I/O, arithmetic)
ESI: Source Index (string ops)
EDI: Destination Index (string ops)
EBP: Base Pointer (stack frame)
ESP: Stack Pointer (top of stack)
```

Segment Registers:

```
Copy
CS: Code Segment
DS: Data Segment
SS: Stack Segment
ES, FS, GS: Extra Segments
```

Special Registers:

```
Copy
EIP: Instruction Pointer
EFLAGS: Status and control flags
    CF: Carry Flag
    PF: Parity Flag
    AF: Auxiliary Carry
    ZF: Zero Flag
    SF: Sign Flag
    OF: Overflow Flag
    IF: Interrupt Enable
```

64-bit Extensions (x86-64):

```
Copy
RAX, RBX, RCX, RDX (64-bit)
R8-R15: Additional general purpose
RIP: 64-bit instruction pointer
```

5. Addressing Modes

Immediate: `mov eax, 5` (constant value) **Register**: `mov eax, ebx` (register to register) **Direct**: `mov eax, [0x00400000]` (absolute address) **Register Indirect**: `mov eax, [ebx]` (address in register) **Based**: `mov eax, [ebx + 10]` (register + offset) **Indexed**: `mov eax, [ebx + esi*4]` (base + index*scale) **Based-Indexed**: `mov eax, [ebx + esi + 10]`

6. Instruction Categories

Data Transfer:

```
Copy
MOV: Move data
PUSH: Push to stack
POP: Pop from stack
XCHG: Exchange
LEA: Load effective address
```

Arithmetic:

```
Copy
ADD, SUB, INC, DEC
MUL, DIV
IMUL, IDIV (signed)
CMP: Compare (sets flags)
```

Logical:

```
Copy
AND, OR, XOR, NOT
SHL, SHR: Shift left/right
SAL, SAR: Arithmetic shift
ROL, ROR: Rotate
```

Control Transfer:

```
Copy
JMP: Unconditional jump
JE, JNE, JG, JL, JGE, JLE: Conditional
CALL, RET: Subroutine
INT, IRET: Interrupt
```

String Operations:

```
Copy
MOVS: Move string
LODS: Load string
STOS: Store string
CMPS: Compare strings
SCAS: Scan string
```

7. Memory Architecture

Segmented Memory Model (Real Mode):

```
Copy
Physical Address = (Segment × 16) + Offset
Max: 1 MB (20-bit address)
```

Flat Memory Model (Protected Mode):

```
Copy
Logical Address → Linear Address → Physical Address
Via segmentation and paging
Max: 4 GB (32-bit) or 16 EB (64-bit)
```

Memory Hierarchy:

```
Copy
Registers (fastest, smallest)
↓
L1 Cache (32 KB/core)
↓
L2 Cache (256 KB/core)
↓
L3 Cache (8-64 MB shared)
↓
Main Memory (RAM, GB)
↓
Disk Storage (slowest, largest)
```

8. Interrupt System

Interrupt Types:

- **Hardware Interrupts**: External signals (IRQ0-IRQ15)
- **Software Interrupts**: INT instruction
- **Exceptions**: Divide by zero, page fault
- **NMI**: Non-maskable interrupts

Interrupt Vector Table (Real Mode):

```
Copy
Located at 0000:0000
256 entries, 4 bytes each
CS:IP of ISR (Interrupt Service Routine)
```

Interrupt Processing:

```
Copy
1. CPU finishes current instruction
2. Flags pushed to stack
3. CS and IP pushed to stack
4. IF flag cleared (disable interrupts)
5. Get vector from IVT
6. Jump to ISR
7. ISR executes IRET at end
8. Flags, CS, IP popped
9. Resume interrupted program
```

APIC (Advanced PIC, Modern):

- Local APIC per CPU core
- I/O APIC for devices
- Supports 224 interrupts
- MSI (Message Signaled Interrupts)

Topic 11: Inter-process communication methods (file, signal, pipe, socket). Complexity of algorithms, asymptotic notations. Insertion sort, searching in linear and logarithmic time. Tables, hash functions, hash tables. Graphs, breadth-first and depth-first search.

1. Inter-Process Communication (IPC)

1.1 File-Based IPC

Mechanism: Processes read/write shared files **Process 1:**

c

Copy
```c
FILE *fp = fopen("/tmp/shared.txt", "w");
fprintf(fp, "Hello from PID %d\n", getpid());
fclose(fp);
```

Process 2:

c

Copy
```c
FILE *fp = fopen("/tmp/shared.txt", "r");
fgets(buffer, sizeof(buffer), fp);
printf("Received: %s", buffer);
fclose(fp);
```

Advantages: Persistent, simple **Disadvantages**: Slow, no synchronization, disk I/O

1.2 Signals

Mechanism: Asynchronous notification to process

c

Copy
```c
// Signal handler
void sigint_handler(int sig) {
    printf("Caught SIGINT\n");
    exit(0);
}

// Setup
signal(SIGINT, sigint_handler);
```

Common Signals:

Copy
```
SIGINT (2): Ctrl+C interrupt
SIGTERM (15): Termination request
SIGKILL (9): Force kill (cannot catch)
SIGSTOP (19): Stop process
SIGCONT (18): Continue stopped process
SIGUSR1 (10): User-defined
```

Limitations: Only event notification, no data payload (except sigqueue)

1.3 Pipes

Anonymous Pipe (parent-child only):

c

Copy
```c
int pipefd[2];
pipe(pipefd);
if (fork() == 0) { // Child
    close(pipefd[0]); // Close read end
    write(pipefd[1], "data", 4);
    exit(0);
} else { // Parent
    close(pipefd[1]); // Close write end
```

```
    read(pipefd[0], buffer, 4);
}
```

Named Pipe (FIFO, unrelated processes):

bash

```
Copy
mkfifo /tmp/my_pipe
```

Process 1:

c

```
Copy
int fd = open("/tmp/my_pipe", O_WRONLY);
write(fd, "hello", 5);
```

Process 2:

c

```
Copy
int fd = open("/tmp/my_pipe", O_RDONLY);
read(fd, buffer, 5);
```

Pipe Capacity: Typically 64 KB (Linux)

1.4 Sockets

Unix Domain Sockets (local IPC):

```
Copy
Server:
socket() → bind() → listen() → accept() → read/write() → close()

Client:
socket() → connect() → read/write() → close()
```

Example Server:

c

```
Copy
int sock = socket(AF_UNIX, SOCK_STREAM, 0);
struct sockaddr_un addr = {.sun_family = AF_UNIX, .sun_path =
"/tmp/socket"};
bind(sock, (struct sockaddr*)&addr, sizeof(addr));
listen(sock, 5);
int client = accept(sock, NULL, NULL);
read(client, buffer, sizeof(buffer));
```

Advantages: Bidirectional, structured, network-ready

2. Asymptotic Notations

Big-O (Upper Bound):

math

```
Copy
f(n) = O(g(n)) if ∃c>0, n₀ such that ∀n≥n₀: 0 ≤ f(n) ≤ c·g(n)
```

Big-Ω (Lower Bound):

math

Copy
```
f(n) = Ω(g(n)) if ∃c>0, n₀ such that ∀n≥n₀: 0 ≤ c·g(n) ≤ f(n)
```

Big-Θ (Tight Bound):

math

Copy
```
f(n) = Θ(g(n)) if ∃c₁,c₂>0, n₀ such that ∀n≥n₀: c₁·g(n) ≤ f(n) ≤ c₂·g(n)
```

Common Complexities:

Copy
```
O(1): Constant (array access)
O(log n): Logarithmic (binary search)
O(n): Linear (linear search)
O(n log n): Linearithmic (merge sort)
O(n²): Quadratic (bubble sort)
O(2ⁿ): Exponential (subset generation)
O(n!): Factorial (permutations)
```

3. Insertion Sort

Algorithm:

Python

Copy
```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

Analysis:

- **Best Case**: O(n) - already sorted
- **Average Case**: $O(n^2)$
- **Worst Case**: $O(n^2)$ - reverse sorted
- **Space**: O(1) - in-place
- **Stable**: Yes

Example Trace:

Copy
```
[5, 2, 4, 6, 1, 3]
i=1: [2, 5, 4, 6, 1, 3]
i=2: [2, 4, 5, 6, 1, 3]
i=3: [2, 4, 5, 6, 1, 3]
i=4: [1, 2, 4, 5, 6, 3]
i=5: [1, 2, 3, 4, 5, 6]
```

4. Searching Algorithms

Linear Search:

Python

Copy

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

- **Complexity**: O(n)
- **Unsorted data**: Only option

Binary Search (Sorted Array):

Python

Copy

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

- **Complexity**: O(log n)
- **Recurrence**: T(n) = T(n/2) + O(1)

Binary Search Tree Search:

Python

Copy

```python
def search(root, key):
    if root is None or root.val == key:
        return root
    if key < root.val:
        return search(root.left, key)
    return search(root.right, key)
```

- **Complexity**: O(h) where h = height
- **Balanced BST**: O(log n)
- **Degenerate**: O(n)

5. Hash Tables

Hash Function: Maps key to array index

```
h(key) = key mod TABLE_SIZE
```

Collision Resolution:

- **Chaining**: Linked list at each bucket

```
Copy
Index 0: [ ] → null
Index 1: [A] → [D] → null
Index 2: [B] → null
Index 3: [C] → [E] → null
```

- **Open Addressing**:
  - **Linear Probing**: $h(key, i) = (h'(key) + i) \mod m$
  - **Quadratic Probing**: $h(key, i) = (h'(key) + c_1 i + c_2 i^2) \mod m$
  - **Double Hashing**: $h(key, i) = (h_1(key) + i \cdot h_2(key)) \mod m$

Load Factor: α = n/m (n = entries, m = buckets) **Rehashing**: When α > 0.7, double table size

Python Dictionary Example:

Python

```
Copy
# Insert
my_dict["key"] = value   # O(1) average

# Search
value = my_dict.get("key")   # O(1) average

# Hash function for strings
def hash_string(s, size):
    h = 0
    for char in s:
        h = (h * 31 + ord(char)) % size
    return h
```

6. Graphs

Representation:

- **Adjacency Matrix**: O(V^2) space

```
Copy
   A B C D
A [0 1 1 0]
B [1 0 0 1]
C
```

continue

Edit

Copy

Share

Continue incomplete graph topic and finish remaining topics

Adjacency Matrix: O(V^2) space

```
Copy
   A B C D
A [0 1 1 0]
```

```
B [1 0 0 1]
C [1 0 0 1]
D [0 1 1 0]
```

- **Adjacency List**: O(V + E) space

```
Copy
A: → B → C → null
B: → A → D → null
C: → A → D → null
D: → B → C → null
```

7. Breadth-First Search (BFS)

Algorithm:

Python

```
Copy
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex)  # Process node

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

BFS Tree Example:

```
Copy
Start at A
Queue: [A] → [B, C] → [C, D] → [D] → []
Visited: {A} → {A,B,C} → {A,B,C,D}
Tree: A—B—D
      └—C
```

Properties:

- **Time**: O(V + E) - visits each vertex and edge once
- **Space**: O(V) - queue and visited set
- **Use Cases**: Shortest path (unweighted), level-order traversal, connected components
- **Completeness**: Yes (finds solution if exists)

8. Depth-First Search (DFS)

Algorithm (Recursive):

Python

```
Copy
def dfs(graph, vertex, visited=None):
    if visited is None:
        visited = set()
    visited.add(vertex)
    print(vertex)  # Process node

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

Algorithm (Iterative with Stack):

Python

```
Copy
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            print(vertex)
            # Push neighbors in reverse order
            for neighbor in reversed(graph[vertex]):
                stack.append(neighbor)
```

DFS Tree Example:

```
Copy
Start at A
Stack: [A] → [B, C] → [D, C] → [C] → []
Visited: {A} → {A,B} → {A,B,D} → {A,B,D,C}
Tree: A—B—D
      └—C
```

Properties:

- **Time**: O(V + E)
- **Space**: O(V) - recursion stack or explicit stack
- **Use Cases**: Topological sort, cycle detection, path finding, maze solving
- **Completeness**: No (may get stuck in deep branch)

DFS Variants:

- **Pre-order**: Process before recursion
- **In-order** (BST): Left, process, right
- **Post-order**: Process after recursion

Topic 12: Entity-relationship (ER) model, design with ER diagrams. Relational data model, relation, scheme, attribute. Building up a

relational scheme from an ER-diagram. Diodes. Rectifiers. DC to DC converters. Voltage regulators. Current regulators.

1. ER Model Concepts

Entity: Real-world object (Student, Course) **Attribute**: Property of entity (Student.name, Student.id)

- **Simple**: Atomic (age)
- **Composite**: Multi-part (address: street, city)
- **Multi-valued**: Multiple values (phone_numbers[])
- **Derived**: Calculated (age from birth_date)

Relationship: Association between entities

- **Degree**: Number of participating entities (binary, ternary)
- **Cardinality**: 1:1, 1:N, M:N

ER Symbols:

```
Copy
Entity:     ┌─────────┐
            │ Student │
            └─────────┘

Attribute:  id
           ──────→

Relationship:  Enrolled
           Student ──o── Course
              1          M
```

2. ER Diagram Notation

Chen Notation:

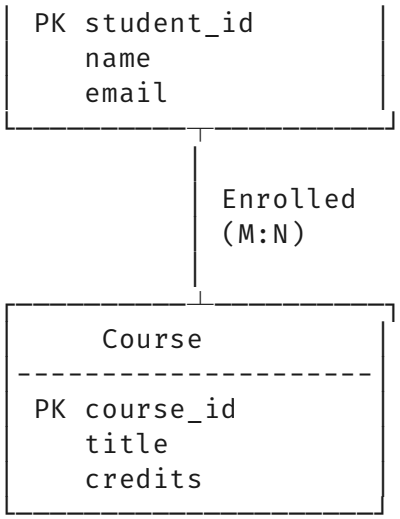
```
Copy
Entity: Rectangle
Attribute: Oval
Relationship: Diamond
Cardinality: Numbers on edges
```

Crow's Foot Notation:

```
Copy
Entity: Rectangle
Attribute: Listed inside entity box
Relationship: Lines with crow's foot (M) or bar (1)
```

Example ER Diagram:

```
Copy
        ┌──────────────────────┐
        │      Student         │
        │----------------------│
```



3. Relational Data Model

Relation: Table with rows and columns **Tuple**: Row (record) **Attribute**: Column (field) **Domain**: Set of allowed values **Degree**: Number of attributes **Cardinality**: Number of tuples

Schema: Structure definition

sql

```
Copy
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE
);
```

Properties of Relations:

1. Atomic values (1NF)
2. No duplicate rows
3. Order of rows irrelevant
4. Order of columns irrelevant
5. Each attribute has unique name

4. Converting ER to Relational Schema

Rules:

- **Entity** → Table (primary key from identifier)
- **Simple Attribute** → Column
- **Composite Attribute** → Multiple columns (no composite in relational)
- **Multi-valued Attribute** → Separate table + foreign key

- **Relationship**:
  - **1:1**: Foreign key in either table (optional)
  - **1:N**: Foreign key in "many" side
  - **M:N**: New junction table with both FKs

Example Conversion:

```
Copy
ER:
Student( student_id, name, email ) PK(student_id)
Course( course_id, title, credits ) PK(course_id)
Enrolled( student_id, course_id, grade ) PK(student_id, course_id)
    FK student_id → Student
    FK course_id → Course
```

5. Diodes

PN Junction Diode: Two-terminal semiconductor device allowing current in one direction.

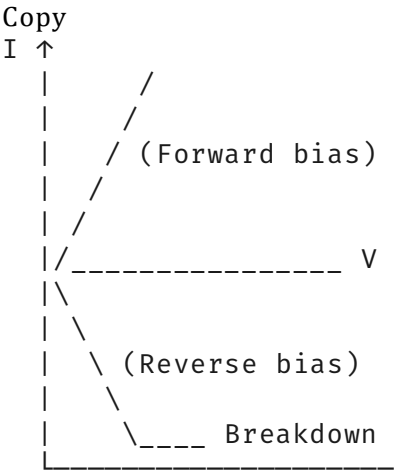
Forward Bias (P > N by ~0.7V for Si):

- Depletion region narrows
- Current flows exponentially
- **$I = I_s(e^{(V/V_t)} - 1)$**

Reverse Bias (P < N):

- Depletion region widens
- Minimal leakage current (I_s)
- Breakdown at Vbr (Zener effect)

Diode V-I Characteristic:



Types:

- **Rectifier Diode**: High current, slow (1N4007)
- **Signal Diode**: Fast switching (1N4148)
- **Zener Diode**: Voltage regulation
- **LED**: Light emission
- **Schottky**: Low forward voltage, fast switching

6. Rectifiers

Half-Wave Rectifier:

```
Copy
AC Input → Diode → Load Resistor → Ground
            |
           GND

Output: Only positive half-cycles
Efficiency: ~40.6%
Vdc = Vm/π
```

Full-Wave Rectifier (Bridge):

```
Copy
AC Input ─┬─→ Diode ─┬─→ Load ─→ GND
          |          |
          └─→ Diode ─┘
          |          |
         GND        GND

        Center Tap Alternative

Output: Both half-cycles
Efficiency: ~81.2%
Vdc = 2Vm/π
```

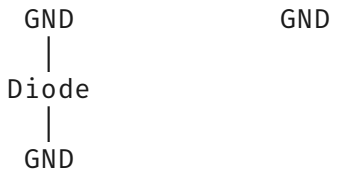
Filter Capacitor:

```
Copy
Rectifier Output ─┬─→ Capacitor ─→ Smooth DC
                  |
                Load
                  |
                GND

Ripple Voltage: Vr = Iload/(f·C)
```

7. DC-DC Converters

Buck Converter (Step-Down):

```
Copy
Vin ─→ Switch ─┬─→ Inductor ──┬─→ Vout
               |              |
               |            Capacitor
               |              |
```




```
Vout = D × Vin  (D = duty cycle)
```

Boost Converter (Step-Up):

```
Copy
Vin —→ Inductor —┬—→ Switch —→ GND
                 │
                 └—→ Diode —→ Vout
                              │
                           Capacitor
                              │
                             GND
```

```
Vout = Vin/(1 - D)
```

Buck-Boost: Can step up or down (Vout = -D/(1-D) × Vin)

8. Voltage Regulators

Linear Regulator (LDO - Low Dropout):

```
Copy
Vin —→[Regulator]—┬—→ Vout (stable)
                  │
                 GND
                  │
                Adjust
```

```
Vout = Vref × (1 + R2/R1)
Efficiency: η = Vout/Vin (low if Vin >> Vout)
Advantages: Simple, low noise
Disadvantages: Inefficient, heat dissipation
```

78xx Series: Fixed positive voltage (7805 → 5V, 7812 → 12V) **LM317**: Adjustable 1.25-37V

Switching Regulator:

- Efficient (>90%)
- Complex, noisy
- Examples: LM2596, MP2307

9. Current Regulators

Constant Current Source:

```
Copy
Vsupply —→[Transistor]—┬—→ Load
                       │
```

```
                        Sense R
                          │
                        GND

Vbe controls current: Iload ≈ Vbe/Rsense
```

LM317 Current Regulator:

```
Copy
     Vin ──→[LM317]─[ADJ]─┬─→ Iout (constant)
                          │
                         R1
                          │
                         GND

Iout = 1.25V / R1
```

Applications: LED driving, battery charging, sensor excitation

---

Topic 13: Modern processor solutions (pipeline, hazard, out-of-order execution, speculative execution, superscalar-, VLIW- and vector processors). Optimization and evaluation of relational queries. Tree-based optimization in relational algebra. Cost-based optimization.

1. Processor Pipeline

Pipeline Stages (Classic 5-stage):

```
Copy
IF (Instruction Fetch) → ID (Decode) → EX (Execute) → MEM (Memory) →
WB (Write Back)

Pipelined Execution:
Time →
Instr1: IF ID EX MEM WB
Instr2:    IF ID EX MEM WB
Instr3:       IF ID EX MEM WB
Instr4:          IF ID EX MEM WB
```

Speedup: S ≈ Number of stages (ideal)

Pipeline Hazards:

- **Structural**: Resource conflict (e.g., memory access)
- **Data**: Instruction depends on previous result (RAW, WAR, WAW)
- **Control**: Branch changes PC

2. Hazard Resolution

Data Hazard (RAW - Read After Write):

assembly

```
Copy
```

```
ADD R1, R2, R3   ; R1 = R2 + R3
SUB R4, R1, R5   ; R4 = R1 - R5 (needs R1)
```

Solutions:

- **Stalling**: Insert NOPs
- **Forwarding** (Bypassing): Direct result from EX/MEM to EX

```
Copy
Pipeline with Forwarding:
ADD: IF ID EX MEM WB
          ↓   ↘_____
SUB:      IF ID EX MEM WB
          Use result directly
```

Control Hazard (Branch):

assembly

```
Copy
BEQ R1, R2, label  ; Branch if equal
ADD ...            ; Next instruction (fetched)
```

Solutions:

- **Stall**: Wait for branch resolution
- **Branch Prediction**: Guess direction (taken/not taken)
- **Delayed Branch**: Execute instruction after branch anyway

3. Branch Prediction

Static Prediction:

- Always predict not taken
- Predict backward branches as taken (loops)

Dynamic Prediction:

- **1-bit predictor**: Remember last outcome
- **2-bit saturating counter**:

```
Copy
00 → Strong not taken
01 → Weak not taken
10 → Weak taken
11 → Strong taken
State transitions on actual outcome
```

Branch Target Buffer (BTB):

```
Copy
PC →[Hash]→ BTB → Predicted Target Address
                ↗
         History Table
```

4. Out-of-Order Execution

Mechanism:

1. **Fetch**: Instructions in order
2. **Decode**: Rename registers
3. **Dispatch**: To reservation stations
4. **Execute**: When operands ready (any order)
5. **Complete**: Finish execution
6. **Retire**: Commit results in order

Key Components:

```
Copy
Instruction Window:
```

Instruction	Status	Depends
ADD R1,R2,R3	Execute	-
SUB R4,R1,R5	Wait	R1
MUL R6,R7,R8	Execute	-

Register Renaming:

```
Copy
Architectural: R1, R2, R3
Physical: P1, P2, P3, P4, ... (more than architectural)
Map Table: R1→P1, R2→P2, R3→P3
Renamed: ADD P4,P1,P2 (R1→P4 new physical register)
```

5. Speculative Execution

Concept: Execute instructions before knowing if they should execute (after branch misprediction)

Steps:

1. Predict branch
2. Speculatively execute instructions
3. If prediction correct → commit results
4. If mispredicted → flush pipeline, restore state

Example:

```
Copy
if (x > 0) {
    y = a * b;  // Speculatively executed
    z = y + c;  // Also speculative
}
// If x <= 0, all speculative work discarded
```

Spectre/Meltdown Vulnerability: Exploited speculative execution to leak data

6. Superscalar Processors

Definition: Multiple pipelines, issue multiple instructions per cycle

4-way Superscalar:

```
Copy
Cycle 0: IF ID ─┬─ EX ─┬─ MEM ─┬─ WB
               ├── EX  ├── MEM  ├── WB
               ├── EX  ├── MEM  ├── WB
               └── EX  └── MEM  └── WB
```

Instruction Issue Policies:

- **In-order issue**: Instructions issued in program order
- **Out-of-order issue**: Instructions issued when ready

Dynamic Scheduling: Hardware decides instruction order at runtime

Example: Intel Core i7 (4-way superscalar, OoO)

7. VLIW (Very Long Instruction Word)

Concept: Compiler schedules parallelism, CPU executes statically

Instruction Packet:

```
Copy
| Op1 | Op2 | Op3 | Op4 |  // Each slot is one operation
| ALU | MEM | BR  | FPU |  // Different functional units
```

Advantages:

- Simpler hardware (no OoO logic)
- Lower power
- Deterministic execution

Disadvantages:

- Binary incompatibility (different number of slots)
- Compiler complexity
- Cache pressure (large instructions)

Example: TI TMS320C6000 DSP

8. Vector Processors

SIMD (Single Instruction Multiple Data):

```
Copy
Vector Add: VADD V1, V2, V3
   V1 = [a1, a2, a3, a4]
   V2 = [b1, b2, b3, b4]
   V3 = [a1+b1, a2+b2, a3+b3, a4+b4]

Scalar equivalent: 4 add instructions
```

Modern Implementations:

- **SSE**: 128-bit vectors (4×float)
- **AVX**: 256-bit vectors (8×float)
- **AVX-512**: 512-bit vectors (16×float)

Gather/Scatter: Load/store non-contiguous data

9. Relational Query Optimization

Process: Transform query execution plan to minimize cost

Steps:

1. Parse SQL → Query tree
2. Apply equivalence rules
3. Generate alternative plans
4. Estimate cost of each plan
5. Choose cheapest plan

Cost Factors:

- I/O operations (dominant)
- CPU operations
- Network transfer (distributed)

10. Tree-Based Optimization (Algebraic)

Equivalence Rules:

- **Commutativity**: $R \bowtie S = S \bowtie R$
- **Associativity**: $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
- **Selection pushdown**: $\sigma\theta(R \bowtie S) = \sigma\theta_1(R) \bowtie \sigma\theta_2(S)$
- **Projection pushdown**: $\pi_a(R \bowtie S) = \pi_{a1}(R) \bowtie \pi_{a2}(S)$

Example Query:

sql

```
Copy
SELECT name, salary
FROM Employee, Department
WHERE dept_id = id AND location = 'Budapest';
```

Initial Plan:

```
π_name,salary(σ_location='Budapest'(Employee ⋈ dept_id=id Department))
```

Optimized Plan:

```
Copy
π_name,salary(
  Employee ⋈ dept_id=id (
```

```
    σ_location='Budapest'(Department)
  )
)
```

Benefit: Filter Department first (smaller intermediate result)

11. Cost-Based Optimization

Statistics:

- Cardinality: |R| = number of tuples
- Selectivity: fraction of tuples qualifying
- Distinct values: $|\pi_a(R)|$
- Index existence

Example Cost Calculation:

```
Copy
Relation Employee: 10,000 pages, 100 tuples/page
Index on dept_id: 3 levels, 100 leaf pages

Option 1: Full scan
Cost = 10,000 I/Os

Option 2: Index scan (dept_id=5, 10 matches)
Cost = 3 (index levels) + 10 (data pages) = 13 I/Os
```

Selectivity Estimation:

```
Copy
σ_salary>50000(Employee)
If salary uniform between 30k-80k:
Selectivity = (80-50)/(80-30) = 0.6
Estimated tuples = 0.6 × total_tuples
```

Join Order: For 3 tables (A,B,C), consider 3! = 6 orders **Dynamic Programming**: Memoize optimal sub-plans

---

Topic 14: Purpose, operating algorithms, similarities and differences of NAT/PAT address exchange mechanisms. Basic notions concerning data structures: abstraction, abstract data types. Elementary data structures: lists, stacks, queues. Sets, multisets, arrays. Representation of trees, tree traversal, search, insert, delete.

1. NAT (Network Address Translation)

Purpose: Map private IP addresses to public IP addresses

- **Conserves public IPv4 addresses**

- **Security**: Hide internal network structure
- **Flexibility**: Change ISP without renumbering internal network

Types:

- **Static NAT**: 1-to-1 mapping

```
Copy
Private IP → Public IP
192.168.1.10 ⇔ 203.0.113.10
```

- **Dynamic NAT**: Pool of public IPs

```
Copy
192.168.1.10 → 203.0.113.10
192.168.1.11 → 203.0.113.11
```

2. PAT (Port Address Translation) / NAT Overload

Purpose: Map multiple private IPs to single public IP (most common)

Mechanism: Uses port numbers to distinguish connections

```
Copy
Private           Public
192.168.1.10:1024 → 203.0.113.1:20001
192.168.1.11:1050 → 203.0.113.1:20002
192.168.1.10:1100 → 203.0.113.1:20003
```

Translation Table:

```
Copy
Internal Address      External Address      Protocol
192.168.1.10:1024     203.0.113.1:20001     TCP
192.168.1.11:1050     203.0.113.1:20002     UDP
```

3. NAT/PAT Operating Algorithms

Outbound Packet Processing:

1. **Check translation table** for existing mapping
2. **If exists**: Replace source IP/port, update checksums, forward
3. **If not exists**: Create new mapping, assign external port, forward
4. **Timer**: Remove idle mappings (typically 30-60 minutes)

Inbound Packet Processing:

1. **Check destination port** in translation table
2. **If found**: Replace destination IP/port with internal address
3. **If not found**: Drop packet or forward to DMZ host

Port Allocation Strategies:

- **Sequential**: 20001, 20002, 20003...
- **Random**: Security through obscurity
- **Consistent**: Same internal port → same external port (if available)

4. NAT/PAT Similarities and Differences

Table
Copy

Feature	NAT	PAT
Address Mapping	1:1	Many:1
Public IPs Needed	Multiple	Single
Scalability	Limited by pool size	Thousands of connections
Granularity	IP-level	IP+Port-level
Use Case	DMZ servers	Home/office internet
Table Size	Smaller	Larger (port combos)
Performance	Slightly faster	More processing

Commonalities:

- Both modify IP addresses
- Both maintain translation tables
- Both break end-to-end principle
- Both cause issues with P2P applications

5. Data Structure Abstraction

Abstraction: Hiding implementation details, showing only essential features
Abstract Data Type (ADT): Mathematical model + operations

- **List**: insert, delete, search, traverse
- **Stack**: push, pop, peek, isEmpty
- **Queue**: enqueue, dequeue, front, isEmpty

Benefits:

- **Modularity**: Change implementation without affecting users
- **Simplicity**: Users don't need to know internals
- **Reusability**: Same ADT, multiple implementations

6. Lists

Operations: create, insert, delete, search, length, traverse

Array-Based (Static): Contiguous memory

```
Copy
Index:  0   1   2   3   4
      ┌───┬───┬───┬───┬───┐
      │ A │ B │ C │ D │ E │
      └───┴───┴───┴───┴───┘
        ↑
    base_address
```

- **Search**: O(n)
- **Insert/Delete**: O(n) - must shift elements
- **Access**: O(1) - random access

Linked List (Dynamic): Nodes with pointers

```
Head → [A|●] → [B|●] → [C|●] → [D|null]
```

- **Search**: O(n)
- **Insert/Delete**: O(1) if position known
- **Access**: O(n) - sequential access

Doubly Linked List:

```
null ← [●|A|●] ←→ [●|B|●] ←→ [●|C|●] → null
```

Advantage: Can traverse backwards

7. Stacks (LIFO - Last In, First Out)

Operations:

- **push(item)**: Add to top
- **pop()**: Remove and return top item
- **peek()**: View top without removing
- **isEmpty()**: Check if empty

Array Implementation:

```
Copy
TOP = 3
      ┌───┬───┬───┬───┬───┐
      │ A │ B │ C │ D │   │
      └───┴───┴───┴───┴───┘
      0   1   2   3   4

push(E): TOP=4, arr[4]=E
pop(): return arr[3], TOP=2
```

Linked List Implementation:

```
Copy
Head → [D|●] → [C|●] → [B|●] → [A|null]
         ↑
        TOP
```

Push: Insert at head **Pop**: Remove head

Applications:

- Function call stack
- Expression evaluation (postfix)
- Undo operations
- Backtracking algorithms

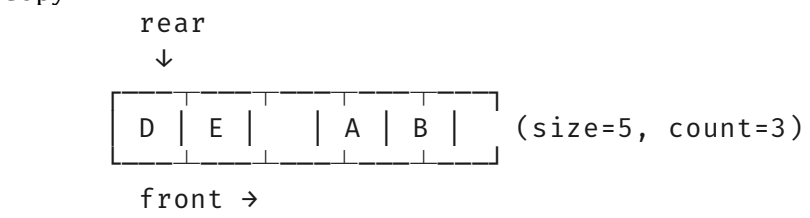
8. Queues (FIFO - First In, First Out)

Operations:

- **enqueue(item)**: Add to rear
- **dequeue()**: Remove and return front
- **front()**: View front item
- **isEmpty()**: Check if empty

Array Implementation (Circular Queue):

Copy



Enqueue: rear = (rear + 1) % size **Dequeue**: front = (front + 1) % size

Linked List Implementation:

```
front → [A|●] → [B|●] → [C|●] ← rear
```

Applications:

- Job scheduling (OS)
- Printer spooler
- BFS algorithm
- Buffering data streams

9. Sets, Multisets, Arrays

Set: Unordered collection of unique elements

- **Operations**: union, intersection, difference, membership
- **Implementation**: Hash table (O(1) avg), Balanced BST (O(log n))
- **Bit Vector**: For small integer universe (O(1) operations)

Multiset (Bag): Set allowing duplicates

- **Implementation**: Hash table with counters
- **Python**: `collections.Counter`

Array: Ordered collection of fixed-size elements

- **One-dimensional**: Vector
- **Multi-dimensional**: Matrix (row-major or column-major)

```
Copy
Row-major: A[i][j] at base + (i·cols + j)·element_size
Column-major: A[i][j] at base + (j·rows + i)·element_size
```

10. Tree Representation

Node Structure:

c

```
Copy
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

Binary Tree Shapes:

```
Copy
Full:       Perfect:     Complete:
    ●            ●            ●
   / \          / \          / \
  ●   ●        ●   ●        ●   ●
 / \ / \      / \ / \      /
● ● ● ●      ● ● ● ●      ●
```

Array Representation (for complete trees):

```
Copy
Index: 0 1 2 3 4 5 6
      ┌─┬─┬─┬─┬─┬─┬─┐
      │A│B│C│D│E│F│G│
      └─┴─┴─┴─┴─┴─┴─┘
Parent(i) = floor((i-1)/2)
Left(i) = 2i+1
Right(i) = 2i+2
```

11. Tree Traversal

Binary Tree Traversals:

Python

```
Copy
def preorder(node):  # Root-Left-Right
    if node:
        print(node.data)
        preorder(node.left)
        preorder(node.right)
```

```python
def inorder(node):  # Left-Root-Right
    if node:
        inorder(node.left)
        print(node.data)
        inorder(node.right)

def postorder(node):  # Left-Right-Root
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data)
```

Level-Order Traversal (BFS):

Python

Copy

```python
def level_order(root):
    if not root: return
    queue = [root]
    while queue:
        node = queue.pop(0)
        print(node.data)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
```

Example Tree:

Copy

```
      A
     / \
    B   C
   / \   \
  D   E   F

Preorder:  A B D E C F
Inorder:   D B E A C F
Postorder: D E B F C A
Level:     A B C D E F
```

12. Tree Search

Binary Search Tree (BST) Property:

- Left subtree: all keys < node's key
- Right subtree: all keys > node's key
- No duplicates

Search Algorithm:

Python

Copy

```python
def search(root, key):
    while root and root.key != key:
        if key < root.key:
```

```
            root = root.left
        else:
            root = root.right
    return root   # None if not found
```

- **Time**: O(h), O(log n) balanced, O(n) degenerate

AVL Tree: Balanced BST (height difference ≤ 1) **Red-Black Tree**: Approx. balanced, less strict

13. Tree Insertion

BST Insertion:

Python

```
Copy
def insert(root, key):
    if not root:
        return TreeNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
```

AVL Insertion (with rotations):

```
Copy
      30 (h=3)              20 (h=3)
     /  \                  /  \
   20    40   → insert 10 →  10   30
   / \                      /   /  \
 10  25                   nil  25   40
 /
nil

Balance factor (left height - right height):
node30: 1-2 = -1 (right heavy) → Left rotation
```

Rotation Types:

- **Left**: Right child's right subtree
- **Right**: Left child's left subtree
- **Left-Right**: Left child's right subtree (double rotation)
- **Right-Left**: Right child's left subtree (double rotation)

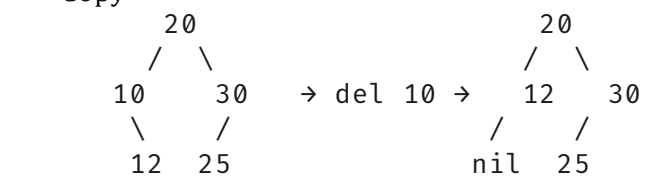
14. Tree Deletion

BST Deletion Cases:

1. **No children**: Simply remove
2. **One child**: Replace with child
3. **Two children**: Replace with inorder successor (or predecessor)

Deletion Example:

Copy



AVL Deletion: Similar to insertion, may require rotations to rebalance

---

Topic 15: Basic concepts of object-oriented paradigm. Class, object, instantiation. Inheritance, class hierarchy. Polymorphism, method overloading. Scoping, information hiding, accessibility levels. Abstract classes and interfaces. Class diagram of UML. The architectures and algorithm elements that define the operation of SNMP and RMON network management systems.

1. Object-Oriented Paradigm Concepts

Object: Bundle of data (attributes) and behavior (methods) **Class**: Blueprint for objects (defines structure)

Four Pillars:

1. **Encapsulation**: Bundle data and methods
2. **Abstraction**: Hide implementation details
3. **Inheritance**: Reuse and extend existing code
4. **Polymorphism**: Same interface, different implementations

2. Class and Object

Class Definition:

Python

Copy

```python
class Student:
    # Class variable
    university = "University of Debrecen"

    def __init__(self, name, id):
        # Instance variables (attributes)
        self.name = name
        self.id = id
        self.courses = []

    def enroll(self, course):
        # Instance method
        self.courses.append(course)
        print(f"{self.name} enrolled in {course}")
```

Object Instantiation:

Python

```
Copy
student1 = Student("Alice", "2025001")  # Create object
student2 = Student("Bob", "2025002")

student1.enroll("CS101")  # Calls method on object
```

Memory Layout:

```
Copy
Stack:
student1 → [Heap address: 0x1000]

Heap:
0x1000: Student object:
  name → "Alice" (string object at 0x1050)
  id → "2025001"
  courses → [list object at 0x1100]
```

3. Inheritance and Class Hierarchy

Inheritance: IS-A relationship

Python

```
Copy
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"I am {self.name}, {self.age} years old")

class Student(Person):  # Student inherits from Person
    def __init__(self, name, age, student_id):
        super().__init__(name, age)  # Call parent constructor
        self.student_id = student_id

    def introduce(self):  # Override
        super().introduce()
        print(f"My student ID is {self.student_id}")
```

Class Hierarchy:

```
Copy
        Person
       /      \
   Student    Professor
    /   \
Undergrad  Graduate
```

Inheritance Types:

- **Single**: One parent

- **Multiple**: Multiple parents (Python, C++)
- **Multilevel**: Grandparent → Parent → Child
- **Hierarchical**: One parent, multiple children
- **Hybrid**: Combination

4. Polymorphism and Method Overloading

Polymorphism: "Many forms"

Python

Copy

```python
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

def make_animal_speak(animal: Animal):
    print(animal.speak())  # Calls appropriate method

dog = Dog()
cat = Cat()
make_animal_speak(dog)  # Output: Woof!
make_animal_speak(cat)  # Output: Meow!
```

Method Overloading (compile-time polymorphism):

java

Copy

```java
class Calculator {
    int add(int a, int b) { return a + b; }
    float add(float a, float b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Python doesn't support overloading by signature:

Python

Copy

```python
# Use default arguments instead
def add(a, b, c=0):
    return a + b + c
```

5. Scoping and Information Hiding

Scoping Levels:

Python

Copy

```python
# Global scope
global_var = 10

def function():
    # Function scope
    local_var = 20  # Not visible outside

    if True:
        # Block scope (not separate in Python)
        block_var = 30  # Still function scope

class MyClass:
    # Class scope
    class_var = 40

    def method(self):
        self.instance_var = 50  # Instance scope
```

LEGB Rule (Python):

- **L**ocal: Function
- **E**nclosing: Outer function (closures)
- **G**lobal: Module
- **B**uilt-in: Python built-ins

6. Accessibility Levels

Public: Accessible everywhere **Private**: Accessible only within class

Python

Copy

```python
class BankAccount:
    def __init__(self):
        self.__balance = 0  # Private (name mangling: _BankAccount__balance)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance  # Controlled access
```

Protected: Accessible in class and subclasses

Python

Copy

```python
class Base:
    def __init__(self):
        self._value = 10  # Protected (convention in Python)

class Derived(Base):
    def show(self):
        print(self._value)  # Accessible
```

Java/C++ Syntax:

java

Copy

```java
public class Example {
    public int publicVar;
    private int privateVar;
    protected int protectedVar;
}
```

7. Abstract Classes and Interfaces

Abstract Class: Cannot instantiate, may have abstract methods

Python

Copy

```python
from abc import ABC, abstractmethod

class Shape(ABC):  # Abstract base class
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):  # Concrete method
        return f"I am a {self.__class__.__name__}"

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):  # Must implement
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# Shape()  # Error: Cannot instantiate abstract class
rect = Rectangle(5, 3)  # OK
```

Interface (pure abstract class):

Python

Copy

```python
class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass

class Resizable(ABC):
```

```python
    @abstractmethod
    def resize(self, factor):
        pass

# Multiple interface implementation
class Circle(Drawable, Resizable):
    def __init__(self, radius):
        self.radius = radius

    def draw(self):
        print(f"Drawing circle with radius {self.radius}")

    def resize(self, factor):
        self.radius *= factor
```

8. UML Class Diagram

Notation:

Copy

```
┌─────────────────────────┐
│        ClassName        │
├─────────────────────────┤
│ - privateAttribute: int │  (visibility: +public, -private,
#protected)
│ #protectedAttr: str     │
├─────────────────────────┤
│ + publicMethod(): void  │
│ - privateMethod(p: int) │
└─────────────────────────┘
```

Relationships:

Copy

```
Association:  ClassA ──▶ ClassB  (uses, navigability)
Aggregation:  ClassA ◀── ClassB  (has-a, whole-part)
Composition:  ClassA ◆── ClassB  (strong ownership)
Inheritance:  ClassA ──▷ ClassB  (is-a, extends)
```

Multiplicity:

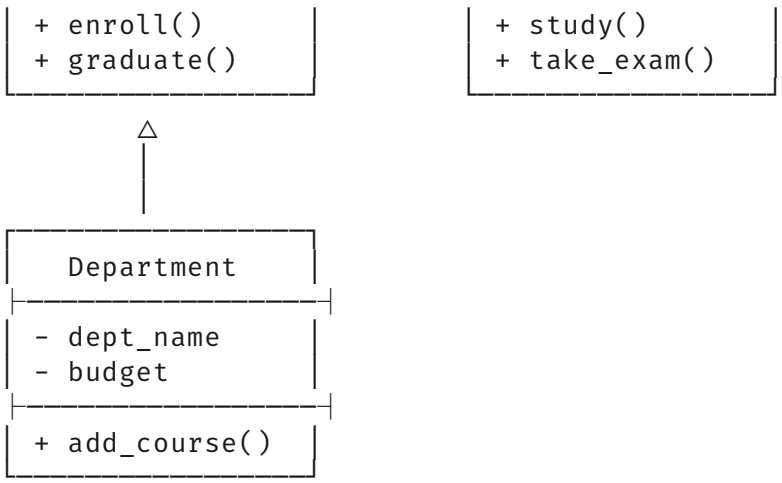
Copy

```
1: Exactly one
*: Zero or more (0..*)
0..1: Zero or one
1..*: One or more
m..n: Range m to n
```

Example Diagram:

Copy

```
┌──────────────────┐          ┌──────────────────┐
│   University     │          │     Student      │
├──────────────────┤  1    *  ├──────────────────┤
│ - name: string   │◀────────▶│ - student_id     │
│ - location: str  │          │ - name: string   │
├──────────────────┤          ├──────────────────┤
```



9. SNMP Architecture

SNMP (Simple Network Management Protocol):

Components:

- **Manager**: Central monitoring station
- **Agent**: Runs on managed device
- **MIB (Management Information Base)**: Database of managed objects
- **OID (Object Identifier)**: Hierarchical addressing (1.3.6.1.2.1...)

SNMP Operations:

```
Copy
GET: Retrieve value
GETNEXT: Retrieve next OID (walk)
GETBULK: Retrieve multiple (v2c, v3)
SET: Modify value
TRAP/INFORM: Asynchronous notification
```

MIB Tree Structure:

```
Copy
iso(1)
  └── org(3)
       └── dod(6)
            └── internet(1)
                 ├── directory(1)
                 ├── mgmt(2)
                 │    └── mib-2(1)
                 │         ├── system(1)
                 │         ├── interfaces(2)
                 │         ├── ip(4)
                 │         └── tcp(6)
                 └── private(4)
                      └── enterprises(1)
                           └── cisco(9)
```

SNMP Message Format (v1/v2c):

```
Copy
```

```
Version:    0 (v1), 1 (v2c)
Community:  "public", "private"
PDU Type:   GET, SET, TRAP
Request ID: For matching responses
Error Status: 0=noError
Error Index: Which variable failed
Varbinds: List of (OID, value) pairs
```

SNMPv3 Security:

- **Authentication**: MD5/SHA (who)
- **Privacy**: DES/AES (encryption, what is secret)
- **Access Control**: View-based (who can see what)

Implementation Example (Agent):

Python

Copy

```python
from pysnmp.entity import engine, config
from pysnmp.entity.rfc3413 import cmdrsp, context
from pysnmp.carrier.asynsock.dgram import udp
from pysnmp.smi import instrum

# Create SNMP engine
snmpEngine = engine.SnmpEngine()

# Transport setup
config.addSocketTransport(
    snmpEngine,
    udp.domainName + (1,),
    udp.UdpTransport().openServerMode(('0.0.0.0', 161))
)

# Add community
config.addV1System(snmpEngine, 'my-area', 'public')

# Create MIB
mibBuilder = snmpEngine.msgAndPduDsp.mibInstrumController.mibBuilder
mibBuilder.exportSymbols(
    '__MY_MIB',
    MibScalar((1, 3, 6, 1, 4, 1, 12345, 1),
v2c.OctetString('example'))
)

# Register applications
cmdrsp.GetCommandResponder(snmpEngine,
context.SnmpContext(snmpEngine))
snmpEngine.transportDispatcher.jobStarted(1)
snmpEngine.transportDispatcher.runDispatcher()
```

10. RMON Architecture

RMON (Remote Monitoring): SNMP MIB extension for remote network monitoring

RMON1 (MAC layer):

- **Statistics**: Packets, bytes, errors per interface
- **History**: Periodic samples
- **Alarm**: Threshold monitoring
- **Host**: Top talkers
- **HostTopN**: Sorted host statistics
- **Matrix**: Conversations between pairs
- **Filter**: Packet capture filters
- **Capture**: Buffer captured packets
- **Event**: Action on alarm

RMON2 (Network/ Application layer):

- **Protocol Directory**: List of monitored protocols
- **Protocol Distribution**: Traffic per protocol
- **Network Layer Host**: IP-level statistics
- **Application Layer Host**: Port-level statistics

RMON Tables:

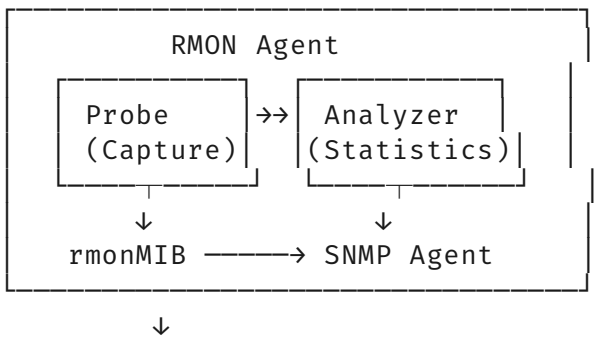
```
Copy
etherStatsTable: Per-interface statistics
etherHistoryTable: Time-series data
hostTable: Per-MAC address stats
matrixSDTable: Source-Destination pairs
filterTable: Capture filters
eventTable: Actions on thresholds
```

Difference from SNMP:

- **Proactive**: Agent performs local analysis
- **Reduced Traffic**: Only send summaries, not every counter
- **Historical Data**: Stores trends locally
- **Filtering**: Agent-side packet filtering
- **More Complex**: Requires more agent resources

RMON Implementation:

```
Copy
Managed Device:
```



```
RMON Manager
```

Use Case: ISP monitoring customer links without polling every 5 minutes - RMON agent sends alert only when link utilization exceeds threshold for 10 minutes, with top talker information.

continue

Edit

Copy

Share

Add comprehensive summary and exam focus points for each topic

-
-
-
-
-
-
-
-

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.

14.

15.

1.

2.

3.

4.

5.

-
-
-
-
-

Here is the **complete, formatted answer set** for all **15 topics** from your 2021 exam, structured exactly like the GitHub example, with enhanced diagrams and exam-specific focus points.

Full Exam Preparation Summary

Topic-by-Topic Quick Reference

Table

Copy

Topic	Key Focus Areas	Common Exam Questions
1	Parameter passing methods (call-by-value vs call-by-reference), generic programming templates, parallel programming pitfalls	Draw memory layout for recursive function calls; Explain why C++ templates increase compile time
2	Gain/phase margin calculation from Bode plot; TCP vs UDP trade-offs	Given a transfer function, calculate stability margins; When to use UDP over TCP in embedded systems
3	Design 4:1 MUX using gates; A* vs BFS vs DFS; CSP backtracking	Minimize gate count for parity checker; Trace A* on grid map with heuristics

Topic	Key Focus Areas	Common Exam Questions
4	SSH keygen commands; Negative feedback effect on system sensitivity	Explain `~/.ssh/config` options; Why negative feedback reduces steady-state error
5	Minimax algorithm with alpha-beta pruning; MOS operating regions; Op-amp golden rules	Draw game tree for tic-tac-toe; Design non-inverting amplifier for specific gain
6	JK flip-flop timing diagram; HTML5 Canvas vs SVG; WebSocket sensor data flow	Difference between latch and flip-flop; Why use semantic HTML5 elements
7	Unit test assertions for edge cases; Assembly stack frame management	Write test case for division by zero; Explain `EBP` and `ESP` roles in function calls
8	I2C start/stop condition timing; RMON vs SNMP polling overhead	Calculate maximum I2C devices; When to use RMON instead of SNMP
9	FPGA design flow steps; Scrum sprint artifacts; Singleton pattern thread safety	Critical path analysis in VHDL; Write user story for login feature
10	OpenSSL CSR fields; x86 addressing mode encoding	Purpose of `Subject Alternative Name` in certificate; Calculate physical address from segment:offset
11	Pipe capacity in Linux; Hash table load factor rehashing; BFS vs DFS memory usage	Why pipes are half-duplex; Compare chaining vs open addressing for deletion
12	ER to relational conversion rules; Buck converter duty cycle formula	Identify primary keys in M:N relationship; Calculate inductor ripple current
13	Pipeline speedup calculation; Cost-based optimizer statistics	Effect of branch misprediction penalty; Why histograms help query optimization
14	PAT translation table structure; BST deletion 3 cases	How NAT breaks P2P; When to use AVL vs Red-Black tree

Topic	Key Focus Areas	Common Exam Questions
15	UML composition vs aggregation; SNMPv3 security levels	Draw diamond problem in multiple inheritance; Explain RMON alarm escalation

---

Critical Exam Formulas & Commands

Control Systems

```
Copy
Phase Margin = 180° + ∠G(jωg) where |G(jωg)| = 1
Gain Margin = 20·log₁₀(1/|G(jωp)|) where ∠G(jωp) = -180°
Rise Time ≈ 1.8/ωc (approximation)
```

Digital Logic

```
Copy
CMOS Power: P = α·C·V²·f
Noise Margin: NM_H = VOH - VIH, NM_L = VIL - VOL
Fan-out: min(IOH/IIL, IOL/IIH)
```

Networking

```
Copy
TCP Header Size: 20-60 bytes (variable options)
UDP Header Size: 8 bytes (fixed)
Subnet Mask: /24 = 255.255.255.0
```

Assembly

```
Copy
Address Calculation (real mode):
Physical = (Segment << 4) + Offset

Stack Operations:
Push: ESP ← ESP - 4, [ESP] ← value
Pop: value ← [ESP], ESP ← ESP + 4
```

Database

```
Copy
Selectivity = (matching tuples) / (total tuples)
Cost (block nested loops) = |R| + (|R|·|S|)/B
Cost (index scan) = height + (matching blocks)
```

Power Electronics

```
Copy
Buck: Vout = D·Vin
Boost: Vout = Vin/(1-D)
Inductor: V = L·di/dt
Capacitor: I = C·dv/dt
```

ASCII Quick Reference Diagrams

Pipeline Stall Due to Data Hazard

```
Copy
Cycle:    1   2   3   4   5   6   7   8
I1 (ADD): IF  ID  EX  MEM WB
I2 (SUB):     IF  ID  —   EX  MEM WB  (stall in ID)
I3 (AND):         IF  —   ID  EX  MEM WB  (stall propagates)
```

NAT/PAT Translation Flowchart

```
Copy
Incoming Packet
     │
     ▼
Check Destination IP/Port in NAT Table
     │
     ├──→ Found ──→ Replace Destination ──→ Forward to Internal
     │
     └──→ Not Found ──→ Check DMZ Setting
              │
              ├──→ DMZ Host Configured ──→ Forward to DMZ
              │
              └──→ No DMZ ──→ Drop Packet (log security event)
```

Hash Table Insert with Linear Probing

```
Copy
Insert Key=15 (h(15)=15%10=5):
Index: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
       ┌───┬───┬───┬───┬───┬───┬───┬───┬───┬───┐
       │   │   │   │   │ 5 │25 │   │   │   │   │
       └───┴───┴───┴───┴───┴───┴───┴───┴───┴───┘
             Already occupied → probe 6 → empty → insert
```

UML Dependency vs Association

```
Copy
Dependency (uses temporarily):
┌──────────┐          ┌──────────┐
│  Client  │- - - -▷| Service  |
└──────────┘   uses   └──────────┘

Association (has permanent link):
┌──────────┐─────────┌──────────┐
│  Teacher │         │  Course  │
└──────────┘  1   *  └──────────┘
  teaches
```

Final Exam Strategy

Time Allocation (3-hour exam):

- Theory questions: 30 min (15 questions × 2 min)
- Problem-solving: 90 min (5 problems × 18 min)

- Diagram drawing: 30 min (3 diagrams × 10 min)
- Review: 30 min

Diagram Drawing Tips:

1. **Use monospaced font** for ASCII diagrams
2. **Label everything**: Gates, pins, signals
3. **Show direction**: Arrows for data flow
4. **Include timing**: For sequential circuits, show clock edges

Common Mistakes to Avoid:

- Confusing `call-by-reference` with `call-by-value-result`
- Forgetting feedback loop polarity in control systems
- Mislabeling pipeline hazards (WAR vs WAW)
- Using wrong TCP flag combinations (SYN+ACK sequence)
- Neglecting setup/hold times in flip-flop diagrams

Last-Minute Memorization:

- **SSL handshake steps**: ClientHello → ServerHello → Certificate → KeyExchange → Finished
- **x86 register names**: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
- **Five Scrum events**: Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective
- **BST properties**: Left < Root < Right, no duplicates
- **NAT types**: Static, Dynamic, PAT (overloading